Libro di Testo — Informatica Classe 3

Corso di Informatica — a.s. 2026/27

Versione 1.0 · 29/10/2026

Corso a cura del prof. Nicola Regge

0 Come lavoriamo (leggere per prime)

1. Si lavora un passo alla volta, con un risultato concreto da vedere subito.
2. Consegne e materiali stanno su Google Classroom; i progetti stanno su GitHub, ognuno nel suo repository.
3. Ogni lezione ha la sua pagina con le schede di teoria, il ripasso e la verifica: leggi le schede prima di rispondere.
4. Se ti blocchi, chiedi all'assistente nella casella della pagina: risponde subito.
5. L'AI aiuta a capire, non a saltare il pensiero: la prova che hai capito è saperlo spiegare a voce con parole tue.
6. Le parti in bangla (una riga di aiuto sotto la frase italiana) sono per chi ne ha bisogno.

1 Le lezioni una per una

In ordine di data: che cosa abbiamo fatto, la teoria e l'esercitazione di ogni lezione.

L01 23/09/2026 — Presentazione dell'anno

Libro parziale: lezioni/2026-09-23_presentazione.md

Che cosa abbiamo fatto

1. **Quando:** 23/09/2026, 2 ore, Laboratorio.
2. **Argomenti:** Presentazione dell'anno scolastico 2026/27: argomenti, regole, modo di lavorare.

La teoria e l'esercitazione

Questa lezione è stata di organizzazione e di avvio: non ha una dispensa scritta. Gli argomenti sono quelli elencati sopra.

L02 25/09/2026 — Creazione delle utenze Git

Libro parziale: lezioni/2026-09-25_utenze-git.md

Che cosa abbiamo fatto

1. **Quando:** 25/09/2026, 2 ore, Laboratorio.
2. **Argomenti:** Account GitHub e che cos'è Git.

La teoria e l'esercitazione

Classe 3 · Informatica · teoria bilingue italiano / বাংলা (bangla)

Italiano

1. **Git** è un **sistema di controllo di versione**: tiene traccia di **tutte le modifiche** ai file di un progetto, nel tempo.
2. **Commit**: una "fotografia" salvata delle modifiche, con un **messaggio** che dice cosa hai cambiato.
3. **Repository** (repo): la **cartella-progetto** con tutti i file e la loro **storia completa**.
4. **A cosa serve:** tornare a una versione precedente, sapere **chi ha cambiato cosa e quando**, lavorare in gruppo senza sovrascriversi.
5. **Git vs GitHub: Git** è lo strumento che tiene la storia (sul tuo computer); **GitHub** è il **sito** che ospita i repository online e permette di condividere e collaborare.
6. **Branch** (ramo): una linea di lavoro **parallela** per provare cose nuove senza rovinare il progetto principale; poi si uniscono (**merge**).
7. **Flusso base:** modifichi i file ⟶ fai un **commit** ⟶ invii online (**push**).

Da ricordare: Git tiene la **storia** del lavoro; un **commit** è un salvataggio con un messaggio; il **repository** è il progetto con la sua storia; **GitHub** è il sito che lo mette online.

বাংলা

1. **Git** হলো একটি **ভার্সন কন্ট্রোল সিস্টেম**: এটি সময়ের সাথে একটি প্রজেক্টের ফাইলের **সব পরিবর্তন** ট্র্যাক করে রাখে।
2. **Commit** (কমিট): পরিবর্তনগুলোর একটি সংরক্ষিত "ছবি", সাথে একটি **বার্তা** যা বলে তুমি কী পরিবর্তন করেছ।
3. **Repository** (রিপোজিটরি): **প্রজেক্ট-ফোল্ডার**, যাতে সব ফাইল এবং তাদের **সম্পূর্ণ ইতিহাস** থাকে।

4. **কেন দরকার:** আগের কোনো সংস্করণে ফিরে যাওয়া, **কে কী ও কখন** পরিবর্তন করেছে তা জানা, এবং একে অপরের কাজ না মুছে দলে কাজ করা।
5. **Git বনাম GitHub: Git** হলো সেই টুল যা ইতিহাস রাখে (তোমার কম্পিউটারে); **GitHub** হলো সেই **ওয়েবসাইট** যা রিপোজিটরি অনলাইনে রাখে ও শেয়ার/সহযোগিতা করতে দেয়।
6. **Branch** (ব্রাঞ্চ): মূল প্রজেক্ট নষ্ট না করে নতুন কিছু চেষ্টা করার একটি **সমান্তরাল** কাজের লাইন; পরে একসাথে জোড়া দেওয়া হয় (**merge**)।
7. **মূল ধাপ:** ফাইল পরিবর্তন করো ⟶ একটি **commit** করো ⟶ অনলাইনে পাঠাও (**push**)।

মনে রাখো: Git কাজের **ইতিহাস** রাখে; একটি **commit** হলো বার্তাসহ একটি সেভ; **repository** হলো ইতিহাসসহ প্রজেক্ট; **GitHub** হলো সেই সাইট যা এটি অনলাইনে রাখে।

L03 28/09/2026 — Git e GitHub; i formati dei file

Libro parziale: lezioni/2026-09-28_git-github-formati.md

Che cosa abbiamo fatto

1. **Quando:** 28/09/2026, 3 ore, Laboratorio.
2. **Argomenti:** Differenza tra Git e GitHub; i formati di file più noti.

La teoria e l'esercitazione

Dispensa facile

Git & GitHub

raccontati con parole semplici

Alla fine di questa dispensa saprai **cos'è Git**, **a cosa serve**, che differenza c'è con **GitHub**, come funziona il **versioning** e vedrai **tanti esempi** di come si usa davvero. Niente comandi da digitare: qui capiamo le idee. Classe 3 · Informatica.

Di cosa parliamo

1. Partiamo da un problema che conosci già **2.** Cos'è Git, in due parole **3.** Un po' di storia: chi l'ha inventato e perché **4.** Git e GitHub non sono la stessa cosa **5.** GitHub è un *servizio*: PaaS, la famiglia …aaS (IaaS/SaaS) e il Freemium **6.** Le parole di Git spiegate facili (repository, commit, branch, merge) **7.** Come si lavora davvero (la storia di due compagni) **8.** Il versioning: i numeri 1.2.3 e come si fa **9.** Tanti casi d'uso con i grafi **10.** Riepilogo e glossario

1. Partiamo da un problema che conosci già

Immagina di dover consegnare una ricerca. La scrivi al computer e, per non perderla, ogni tanto la salvi con un nome nuovo. Dopo qualche giorno sul tuo desktop c'è questa scena:

ricerca.doc · ricerca_v2.doc · ricerca_BUONA.doc · ricerca_DEFINITIVA.doc · ricerca_DEFINITIVA_vera_QUESTA.doc

Ti suona familiare, vero? È un pasticcio. E nascono tre problemi:

1. Non sai più **qual è l'ultima** davvero buona, né **cosa è cambiato** da una all'altra.
2. Se cancelli una frase importante e te ne accorgi dopo, **non torni indietro**.
3. Se tu e un compagno lavorate sullo stesso file e ve lo passate, uno finisce per **cancellare** il lavoro dell'altro senza volerlo.

Ecco: **Git nasce per risolvere esattamente questo**. E lo fa in modo elegante.

2. Cos'è Git, in due parole

Git è come una **macchina del tempo** per i tuoi file. Tu lavori normalmente, e ogni volta che vuoi dici a Git: "*ecco, salva com'è adesso*". Git scatta una **fotografia** e la mette in fila con le altre. Da quel momento puoi **tornare a qualsiasi fotografia del passato** quando vuoi: com'era ieri, la settimana scorsa, un mese fa.

La cosa bella è che tutte queste fotografie stanno dentro **una sola cartella**, ordinata e pulita. Niente più venti file con nomi strani: uno solo, con dentro **tutta la storia**.

In una frase: Git è un **sistema di controllo di versione** — cioè un programma che tiene la storia di un progetto e ti lascia viaggiare avanti e indietro nel tempo.

Esempio. Stai programmando un videogioco. Funziona tutto, poi provi a cambiare una cosa e… si rompe. Con Git non è un dramma: torni all'ultima fotografia che funzionava e riparti da lì. Zero panico.

3. Un po' di storia: chi l'ha inventato e perché

Git non è nato in un ufficio di marketing: è nato da un **bisogno reale**.

Nel **2005** un programmatore famoso, **Linus Torvalds** (lo stesso che aveva creato **Linux**, il sistema operativo), aveva un problema grosso. Al codice di Linux lavoravano **migliaia di persone** sparse per il mondo, tutte insieme. Serviva uno strumento per mettere ordine in quel caos. Quello che usavano prima, a un certo punto, non si poté più usare gratis.

Invece di arrendersi, Torvalds fece una cosa da film: **si scrisse lo strumento nuovo da solo, in pochi giorni**. Doveva essere **velocissimo**, non perdere **mai** i dati, e reggere **tantissime persone** che lavorano in parallelo. Quello strumento è Git.

Curiosità. In inglese "git" è una parola scherzosa per dire "testa dura". Torvalds disse ridendo di dare sempre ai suoi progetti "un nome ispirato a sé stesso". Oggi quel piccolo programma è lo **standard usato in tutto il mondo del lavoro**.

4. Git e GitHub non sono la stessa cosa

Qui all'inizio si confondono tutti, quindi fermiamoci un attimo. **Git** e **GitHub** sembrano gemelli ma fanno due mestieri diversi.

1. **Git** è lo **strumento** (la macchina del tempo di cui parlavamo). Non è di nessuno: è **gratis e "open source"**, cioè il suo codice è pubblico e chiunque può usarlo.
2. **GitHub** è un **sito web** dove metti i tuoi progetti Git **online**, per averli al sicuro, mostrarli e lavorarci in gruppo. Dietro GitHub c'è un'**azienda vera** (GitHub, Inc.), oggi di proprietà di **Microsoft**.

Analogia della macchina: Git è il **motore** — la tecnologia che fa funzionare tutto. GitHub è il **grande garage online** dove parcheggi le tue auto, le fai vedere agli altri e ci lavorate insieme. Il motore puoi usarlo anche senza quel garage; ma il garage è comodo.

E attenzione: GitHub **non è l'unico garage**. Ci sono anche **GitLab** e **Bitbucket**, che fanno cose simili. Noi useremo GitHub perché è il più diffuso — quando cerchi lavoro, quello che quasi tutti conoscono è lui.

5. GitHub è un "servizio": PaaS e Freemium

Abbiamo detto che GitHub è un servizio online. Due parole che hai visto alla lavagna spiegano **che tipo** di servizio è.

PaaS — "la piattaforma come servizio"

PaaS è la sigla di **Platform as a Service**, cioè "piattaforma **come servizio**". Vuol dire: qualcuno ti mette a disposizione online una **piattaforma già pronta**, e tu la usi senza doverti costruire e mantenere i computer, i server e tutto il resto.

Analogia della cucina. Vuoi aprire un ristorante. Puoi **costruirti la cucina da zero** (comprare forni, frigoriferi, sistemare i tubi… tanta fatica), oppure **affittare una cucina già attrezzata** e pensare solo a cucinare. GitHub è la cucina già pronta: tu porti il tuo progetto, il resto lo mette lui.

Freemium — "gratis + premium"

Freemium è una parola inventata unendo **FREE** (gratis) e **PREMIUM** (a pagamento). Significa: le funzioni **di base sono gratis** per tutti, e se vuoi funzioni **in più** paghi un abbonamento.

Esempio che conosci. Tanti videogiochi per telefono sono così: giochi gratis, ma se vuoi lo scudo dorato o niente pubblicità, paghi. Anche Spotify: ascolti gratis con la pubblicità, oppure paghi il "premium" e la togli. **GitHub funziona allo stesso modo:** a te studente basta e avanza la parte gratis.

La famiglia "come servizio" (…aaS)

PaaS non è da solo: fa parte di una **famiglia** di servizi online, tutti con la sigla che finisce in **"aaS"** (*as a Service*, "come servizio"). Cambia **quanto lavoro fai tu** e quanto lo fa il fornitore. I tre principali, dal "fai più tu" al "fai meno tu":

Sigla	Nome	Cosa ti danno già pronto	Esempio
IaaS	Infrastructure as a Service	Solo i **muri e la corrente**: computer/server nudi. Il resto lo monti tu.	Amazon AWS, Google Cloud (server in affitto)
PaaS	Platform as a Service	La **cucina già attrezzata**: piattaforma pronta, tu porti solo il progetto.	GitHub, Netlify
SaaS	Software as a Service	Il **piatto già cucinato e servito**: un programma pronto, lo usi dal browser.	Gmail, Google Documenti, Spotify

Come ricordarli — la pizzeria. IaaS = affitti il **locale vuoto** e ti arrangi. **PaaS** = affitti la **cucina attrezzata** e pensi solo a cucinare la pizza. **SaaS** = ti fai **consegnare la pizza pronta** a casa. Più scendi, meno fatica fai (ma decidi meno cose).

Riassunto della lavagna: GitHub è un **servizio** — nello specifico una **piattaforma** (**PaaS**, uno della famiglia …aaS) — dove tieni **repository di codice**, e si paga col modello **Freemium**: gratis per l'uso normale, a pagamento per le cose avanzate.

6. Le parole di Git spiegate facili

Git ha un suo piccolo vocabolario. Sono solo quattro parole importanti: impariamole con un esempio ciascuna.

Repository (repo)

È la **cartella-progetto** che Git tiene d'occhio, con dentro i file **e tutta la loro storia**. Creare un repository vuol dire dire a Git: "*da qui in poi, di questa cartella, tieni la storia*".

Esempio. Il tuo videogioco "SpaceRun" è un repository: dentro c'è il gioco **e** il diario di tutte le modifiche fatte.

Commit

Un **commit** è una delle nostre **fotografie**: un salvataggio, con attaccato un **messaggio** che dice cosa hai cambiato.

"menu""punteggio""grafica""suoni"

Una fila di commit: ognuno è una foto con un messaggio. Puoi tornare a qualunque punto.

Consiglio: scrivi messaggi chiari ("aggiunto il menu iniziale"). Riletti dopo un mese, sembrano il diario del progetto.

Branch (ramo)

Un **branch** è una **strada parallela**. Serve per provare un'idea nuova **senza rovinare** la versione che funziona (la strada principale si chiama main).

Esempio. Vuoi provare a mettere i colori dark nel gioco, ma non sei sicuro. Apri un branch "colori-scuri", provi lì. Se piace lo tieni; se no, lo butti e il gioco resta com'era.

Merge (unione)

Quando l'idea del branch funziona, la **unisci** alla strada principale: questo si chiama **merge**. Da quel momento anche il main ha la novità.

7. Come si lavora davvero: la storia di due compagni

Aisha e Marco fanno insieme il gioco "SpaceRun" su GitHub. Guardiamo cosa fanno, passo passo:

1. **clone** — la prima volta ognuno **scarica** il progetto dal sito sul proprio computer.
2. **lavora** — Aisha disegna i nemici, Marco scrive i movimenti. Ognuno sul suo pezzo.
3. **commit** — ogni tanto scattano una fotografia del proprio lavoro, con un messaggio.
4. **push** — **mandano** le loro fotografie **online**, su GitHub, così l'altro le vede.
5. **pull** — **scaricano** le novità dell'altro, per restare aggiornati.

Il tuo computer GitHub (online) push ⟶ pull

E se Aisha vuole proporre a Marco una modifica importante, senza imporgliela? Apre una **Pull Request**: è come dire "*ehi, ho fatto questa cosa, la guardi? se ti va la uniamo*". Marco la legge, commenta, e se è d'accordo si fa il merge. Ordinato e senza litigi.

Importante per noi: faremo **tutto dal browser**, cliccando sui bottoni del sito GitHub. Le parole (commit, push, merge…) restano quelle, ma non si digita niente.

8. Il versioning: i numeri 1.2.3 e come si fa

Quando un progetto arriva a un punto "buono", gli si dà un **numero di versione**. Lo standard più usato al mondo ha la forma **v1.2.3**, e ogni numero racconta una cosa:

Numero	Nome	Aumenta quando…
1 (il primo)	major	cambi **tutto**, in modo grosso, magari incompatibile col vecchio
2 (il secondo)	minor	aggiungi una **funzione nuova**, ma il resto funziona ancora
3 (il terzo)	patch	correggi solo un **bug**, niente di nuovo

Esempio con il gioco. v1.0.0 è la prima versione giocabile ⟶ aggiungi i suoni e diventa v1.1.0 ⟶ trovi un bug e lo correggi: v1.1.1 ⟶ rifai il gioco da zero con un motore nuovo: v2.0.0.

Come si fa, in pratica

1. Lavori e fai i tuoi **commit** (le fotografie) come sempre.
2. Quando il progetto è **pronto e provato**, scegli il numero: si parte da v1.0.0.
3. Metti una **bandierina** (si chiama **tag**) su quel commit con scritto il numero.
4. Su GitHub pubblichi una **Release**: quella versione diventa **ufficiale** e **non cambia più**.
5. Continui a lavorare e, alla modifica dopo, **aumenti il numero** secondo la regola sopra.

Regola d'oro: un numero già pubblicato **non si riusa mai**. Se cambi anche una sola cosa, prima **aumenti** il numero e poi pubblichi. Così due versioni diverse non si confondono mai.

9. Tanti casi d'uso con i grafi

Vediamo Git "in azione" in situazioni vere. In ogni grafo ogni **pallino** è un commit (una foto) e la **linea** è il tempo, che va da sinistra a destra.

Caso A — Lavoro da solo, tutto dritto

Stai facendo un piccolo sito da solo. Salvi passo dopo passo: la storia è una **linea dritta**, semplice semplice.

iniziohome pagecontattifinito

Caso B — Provo un'idea rischiosa

Vuoi provare un'idea che **forse non funziona**. Apri un **branch** a parte: se l'idea è buona la **unisci** (merge); se è un disastro **butti il ramo** e il main resta intatto. Nessun rischio.

main

idea nuova (branch)

merge (se funziona)

Caso C — Bug urgente mentre sto lavorando

Stai costruendo una funzione lunga, ma salta fuori un **bug grave** nella versione già pubblicata. Apri un ramo **hotfix** (correzione veloce), sistemi il bug, lo unisci subito al main — e poi torni tranquillo alla tua funzione, senza aver perso niente.

main

hotfix (bug)

la mia funzione lunga (branch)

Caso D — Squadra: tanti rami e Pull Request

Nel progetto di gruppo ognuno lavora sul **suo ramo** (chi la grafica, chi i movimenti, chi l'audio). Ciascuno apre una **Pull Request**; il gruppo la controlla e la unisce. Il main cresce con i pezzi di tutti, senza pestarsi i piedi.

main

grafica (PR)

movimenti (PR)

audio (PR)

Caso E — Le versioni pubblicate (Release)

Mentre il progetto avanza, si pubblicano le **Release**. Le bandierine sul main segnano le versioni ufficiali che gli utenti (o i compagni) ricevono. È esattamente il grafo che abbiamo disegnato alla lavagna.

v1.0v1.1v1.2v2.0

1. suoni+ livellitutto nuovo

10. Riepilogo e glossario

Se ti ricordi solo tre cose, ricordati queste:

1. Git è la **macchina del tempo** del tuo progetto: salva la storia e ti fa tornare indietro.
2. **Git** è lo strumento; **GitHub** è il sito-servizio (una piattaforma **PaaS**, gratis in modello **Freemium**) dove lo tieni online e collabori.
3. Le **versioni** si scrivono v1.2.3 e una pubblicata non si tocca più: se cambi, aumenti il numero.

Parola	Vuol dire
Git	lo strumento (gratis, open source) che tiene la storia del progetto
GitHub	il sito-servizio (azienda di Microsoft) che ospita i progetti online
PaaS	"piattaforma come servizio": una piattaforma già pronta che usi online
Freemium	gratis nelle funzioni base, a pagamento per quelle avanzate
repository	la cartella-progetto con tutta la storia
commit	un salvataggio (una fotografia) con un messaggio
branch	una strada di lavoro parallela (ramo)
merge	unire un ramo alla strada principale
clone	scaricare la prima volta un progetto sul proprio computer
push / pull	mandare / scaricare le fotografie da e verso GitHub
Pull Request	proposta di unione, da guardare prima di fare il merge
tag / Release	una versione pubblicata e "congelata"
versioning	dare un numero (v1.2.3) alle versioni del progetto

Classe 3 · Informatica · v1.1 · si scrive in Google Documenti e si consegna su Google Classroom · italiano

Obiettivo: dimostrare che hai capito **cosa sono i tipi di file** (formati, tag, compressione, video) con parole tue. Conta la comprensione, non le frasi perfette. Puoi usare la dispensa per ripassare, ma le risposte devono essere **tue**.

Ripasso lampo

1. L'**estensione** (.mp4, .md…) è il "cognome" del file: dice il tipo e con che programma si apre.
2. I file **a tag** (HTML, XML, MD) marcano il testo con etichette per dargli struttura.
3. **Refresh rate** = quante volte al secondo si aggiorna lo **schermo** (Hz); **fps** = fotogrammi al secondo di un **video**.
4. **Compressione**: senza perdita (ZIP, PNG = identico) o con perdita (JPG, MP4 = più leggero).

Apri un documento nuovo: nel browser vai su docs.new e dai un **titolo** (es. "I tipi di file — Nome Cognome"). Rispondi alle 5 parti.

Parte 1 — Le basi (con parole tue)

1. Cos’è un **file** e a cosa serve l’**estensione**? (2-3 frasi)
2. Perché rinominare foto.jpg in foto.txt non la trasforma in testo?

Parte 2 — Tre formati diversi

1. Scegli **tre formati** di famiglie diverse (es. uno di testo, uno immagine/video, uno archivio) e per ognuno scrivi: **a cosa serve** e **con che programma si apre**.

Parte 3 — Un file a tag

1. Scrivi un **piccolo esempio** (3-4 righe) di un file a tag a tua scelta (**HTML** o **XML** o **MD**) e spiega cosa fanno i tag che hai usato. Esempio di partenza da modificare:

<h1>Il mio titolo</h1> Un paragrafo con una parola in grassetto.

Parte 4 — Compressione

1. Spiega la differenza tra compressione **senza perdita** e **con perdita**, con **un esempio** per ciascuna (quale formato e in quale situazione la useresti).

Parte 5 — Schermo e video

1. Con parole tue: che differenza c’è tra il **refresh rate** di un monitor (Hz) e gli **fps** di un video? Perché sono due cose diverse ma collegate?

Parte 6 — Testo o binario, e il percorso

1. Come fai a capire se un file è di **testo** o **binario**? Fai **un esempio** di ciascuno.
2. Scrivi il **percorso (path)** di un file d’esempio sul computer (es. C:\Documenti\nomefile.txt).

Quando hai finito, **consegna** il documento su **Classroom**.

Nota: conta soprattutto **saperlo spiegare a voce**, con parole tue.

L04 30/09/2026 — Una pagina web da zero

Libro parziale: lezioni/2026-09-30_pagina-html-locale.md

Che cosa abbiamo fatto

1. **Quando:** 30/09/2026, 3 ore, Laboratorio.
2. **Argomenti:** La prima pagina HTML sul proprio PC: struttura, tag, CSS.

La teoria e l’esercitazione

v1.0

Dispensa 1 — La mia pagina web da zero

Classe 3 · Informatica · 05/10/2026 · HTML + CSS in due file separati · italiano

Cosa ottieni oggi: una cartella **mio-sito** con **due file** : **index.html** (il **contenuto** : i testi) e **style.css** (l’**aspetto** : i colori). Doppio clic e la tua pagina si apre nel browser. Nella Dispensa 2 la mettiamo **su internet**.

Ciao, sono Leo

Questo è il mio primo sito web.

Chi sono

Studio informatica e mi piace creare cose nuove.

Le mie passioni

1. Calcio

2. Videogiochi
3. Musica

Il mio sogno

Lavorare con i computer.

Sito creato da me - 2026

Ecco come sarà (poi la fai tua)

Desktop > mio-sito

index.html style.css

index.html = CONTENUTO
style.css = ASPETTO

La cartella: esattamente due file

Carta e penna: disegna sul quaderno la cartella **mio-sito** con dentro i due file. Fai una freccia da index.html a style.css e scrivici sopra **< link>**. Scrivi: **HTML = contenuto, CSS = aspetto**.

Il tuo sito diventerà PUBBLICO (lo vedrà chiunque su internet). Quindi scrivi **solo il nome o un soprannome**. **MAI** cognome, telefono, email, indirizzo, foto del viso.

1. Il codice (uguale per tutti — poi lo fai tuo)

Il modo più sicuro per copiare: apri la pagina del corso (il link è nel compito su Classroom) e premi il bottone **Copia** sopra il codice. Poi nel Blocco note premi Ctrl + V. Copiare dal PDF a volte rovina gli spazi.

RIQUADRO 1 — file index.html (il contenuto)

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Il mio primo sito</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Ciao, sono Leo</h1>
  <p class="sotto">Questo è il mio primo sito web.</p>

  <div class="scheda">
    <h2>Chi sono</h2>
    <p>Studio informatica e mi piace creare cose nuove.</p>

    <h2>Le mie passioni</h2>
    <ul>
      <li>Calcio</li>
      <li>Videogiochi</li>
      <li>Musica</li>
    </ul>

    <h2>Il mio sogno</h2>
    <p>Lavorare con i computer.</p>
  </div>

  <p class="piede">Sito creato da me - 2026</p>
</body>
</html>
```

RIQUADRO 2 — file style.css (l’aspetto)

```
body {
  background: lightblue;
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 20px;
}
```

```
h1 {
  color: darkblue;
}

.sotto {
  color: gray;
}

.scheda {
  background: white;
  max-width: 500px;
  margin: 20px auto;
  padding: 20px;
  border-radius: 12px;
  text-align: left;
}

h2 {
  color: teal;
}

.piede {
  font-size: 12px;
  color: gray;
}
```

2. I passi, uno alla volta

1Crea la cartella

Sul **Desktop** , in un punto vuoto: clic con il tasto **DESTRO** del mouse -> **Nuovo** -> **Cartella**. Scrivi il nome mio-sito e premi Invio.

2Fai vedere la fine dei nomi (consigliato)

Apri la cartella **mio-sito**. In alto clic su **Visualizza** -> **Mostra** -> **Estensioni nomi file** (compare la spunta). Così vedi se un file finisce con **.html** o, per errore, con **.txt**.

3Apri il Blocco note

In basso a sinistra clic su **Start** , scrivi Blocco note e clicca sul programma **Blocco note**.

4Incolla il codice HTML

Copia **tutto** il **RIQUADRO 1** e incollalo nel Blocco note (Ctrl + V).

5Fallo tuo

Cambia **Leo** con il tuo nome o un soprannome. Cambia **Chi sono** , **le passioni** e **il sogno** con cose tue. Non toccare le parole tra < e >: cambia solo il testo in mezzo.

6Salva come index.html

In alto a sinistra **File** -> **Salva con nome**. A sinistra scegli **Desktop** e apri **mio-sito**. In basso: **Salva come** -> **Tutti i file** ; **Nome file** : index.html; **Codifica** : **UTF-8**. Clic su **Salva**.

Salva con nome

Cartella: **Desktop > mio-sito**

Nome file: index.html

Salva come: Tutti i file (.)

Codifica: UTF-8 Salva

7Un foglio nuovo per il CSS

Nel Blocco note: **File** -> **Nuovo** (si apre un foglio vuoto). Copia **tutto** il **RIQUADRO 2** e incollalo.

8Salva come style.css

Come prima: **File** -> **Salva con nome** , **stessa cartella mio-sito** , **Tutti i file** , nome style.css, **UTF-8** , **Salva**.

9Controlla la cartella

Dentro **mio-sito** ci devono essere **due file** : index.html e style.css. Nomi tutti **minuscoli** , senza spazi, senza **.txt** alla fine.

10Apri la tua pagina

Doppio clic su **index.html** : si apre nel browser, **con i colori**. **FATTO! Hai creato la tua pagina web.**

11Cambia i colori (il CSS)

Clic **destro** su **style.css** -> **Apri con** -> **Blocco note**. Cambia lightblue con un colore che ti piace. Salva (Ctrl + S) e nel browser premi F5: il colore cambia!

Colore	Scrivi	Colore	Scrivi	Colore	Scrivi
	pink		lightgreen		gold
	orange		lavender		coral
	khaki		turquoise		salmon

3. Cosa fa ogni pezzo

Pezzo di codice	Cosa fa
<head> … </head>	istruzioni per il browser (non si vedono)
<link rel=”stylesheet” href=”style.css”>	**collega** l’HTML al file style.css
<body> … </body>	tutto quello che si vede nella pagina
<h1> <h2>	titolo grande, titolo più piccolo
<p>	un paragrafo di testo
	un elenco e le sue voci
	una scatola; il nome **scheda** lo usa il CSS
.scheda	CSS: **chi** { **proprietà** : **valore** ; }

4. Se qualcosa non va (succede a tutti, anche ai professionisti)

Problema	Soluzione
Vedo il **codice** invece della pagina	Il file è stato salvato come **.txt** : risalva con **Tutti i file**.
La pagina è **senza colori**	style.css non è nella stessa cartella, o ha un nome diverso (es. **Style.css** , **stile.css** , **style.css.txt**).
Al posto di **è** vedo simboli strani	Risalva index.html scegliendo **Codifica: UTF-8**.
Ho cambiato ma non vedo niente	Hai salvato? (Ctrl+S) Poi nel browser F5.

Hai già fatto la pagina del 30/09? Benissimo: puoi riusare i tuoi testi. Ma per il sito servono **questi nomi esatti** : **index.html** e **style.css** , e nel deve esserci **style.css**.

Prossimo passo: Dispensa 2 — mettiamo il tuo sito **su internet** , con un indirizzo da aprire anche sul telefono.

Classe 3 · Dispensa 1 · v1.0 · Tutti i dati negli esempi sono inventati.

L05 02/10/2026 — Pagina «Mi presento» con CSS separato

Libro parziale: lezioni/2026-10-02_mi-presento.md

Che cosa abbiamo fatto

1. **Quando:** 02/10/2026, Laboratorio.
2. **Argomenti:** Pagina HTML «Mi presento» con il CSS in un file separato.

La teoria e l’esercitazione

v1.0

La mia pagina web in 7 passi

Classe 3 · scheda FACILE · 05/10/2026 · italiano

Alla fine: una pagina web con il **tuo nome** , che si apre nel browser. Ci vogliono circa 20 minuti. Una cosa alla volta!

1Fai una cartella

Sul Desktop: tasto **destro** del mouse -> **Nuovo** -> **Cartella**. Scrivi mio-sito e premi Invio.

2Copia il codice

Apri il link del compito su Classroom. Sul **RIQUADRO 1** premi il bottone giallo **Copia**.

3Incolla nel Blocco note

Start -> scrivi Blocco note -> aprilo. Premi Ctrl + V.

4Metti il tuo nome

Trova **Leo** e scrivi al suo posto il tuo nome (o un soprannome). Niente cognome!

5Salva: index.html

File -> **Salva con nome** -> cartella **mio-sito**. **Salva come: Tutti i file**. Nome: index.html -> **Salva**.

Salva con nome

Nome file: index.html
Salva come: Tutti i file (.)

6Il secondo file: style.css

File -> **Nuovo**. Copia il **RIQUADRO 2** (bottone Copia) e incolla. Salva come prima, nome style.css.

Desktop > mio-sito

index.html style.css

7Apri la tua pagina

Nella cartella **mio-sito** fai **doppio clic** su **index.html**. **FATTO! È la tua pagina web!**

Bonus: apri style.css con il Blocco note, cambia lightblue con pink o gold, salva e premi F5. **Problema?** Alza la mano: succede a tutti!

L06 05/10/2026 — Il sito pubblico su GitHub Pages

Libro parziale: lezioni/2026-10-05_sito-github-pages.md

Che cosa abbiamo fatto

1. **Quando:** 05/10/2026, Laboratorio.
2. **Argomenti:** Pubblicazione del sito personale su GitHub Pages.

La teoria e l'esercitazione

v1.0

Dispensa 2 — Pubblica il tuo primo sito su GitHub

Classe 3 · Informatica · 05/10/2026 · GitHub Pages, tutto dal browser · italiano

Cosa ottieni: un **indirizzo internet vero** , come https://tuonome.github.io/mio-sito/, che chiunque può aprire, **anche sul telefono**. È il tuo primo sito pubblico.

Prima di iniziare ti serve: 1) la cartella **mio-sito** con **index.html** e **style.css** (Dispensa 1); 2) il tuo account **GitHub** (nome utente e password).
Parole nuove: repository = la cartella del progetto su GitHub · **commit** = salvare una versione · **upload** = caricare · **GitHub Pages** = il servizio gratis che trasforma il repository in un sito.

Controllo privacy PRIMA di caricare: apri index.html e verifica: **niente cognome, telefono, email, indirizzo, foto del viso**. Quello che carichi lo vede il mondo intero.

Se la pagina di GitHub è tradotta (in italiano o nella tua lingua) i nomi dei bottoni cambiano: cerca il bottone per **posizione e colore** , come nei disegni (il cerchio **rosso** indica dove cliccare).

1. Crea il repository

1Entra su GitHub

Apri il browser e vai all'indirizzo github.com. In alto a destra **Sign in** (Accedi): scrivi nome utente e password.

2Nuovo repository

In alto a destra clic sul **+** -> **New repository** (Nuovo repository).

github.com

GitHub+ ▾ <- 1

New repository <- 2

Import repository

New organization

3Dai il nome e crea

In **Repository name** scrivi mio-sito. Lascia **Public** (Pubblico). Metti **Add README** su **On** (o spunta la casella). In basso a destra clic sul bottone **verde Create repository**.

github.com/new

Repository name: mio-sito

◉ **Public** ○ Private

Add README: On

Create repository

2. Carica i tuoi due file

4Add file -> Upload files

Nella pagina del repository clic su **Add file** (bottone **grigio** , a sinistra del bottone verde **Code**) -> **Upload files** (Carica file).

github.com/tuonome/mio-sito

tuonome / mio-sito *Public*

Go to file Add file ▾ <> Code ▾

Create new file

Upload files <-

5Trascina i file

Apri **Esplora file** -> **Desktop** -> **mio-sito**. Seleziona i due file (Ctrl + A) e **trascinali** nel riquadro tratteggiato della pagina. (Oppure clic su **choose your files** e scegli i due file.)

github.com/tuonome/mio-sito/upload/main

Drag files here to add them to your repository
or *choose your files*

index.html style.css

6Salva su GitHub: il commit

Controlla che si vedano **index.html** e **style.css**. In basso clic sul bottone **verde Commit changes**. Hai fatto un **commit** : hai salvato una versione, come abbiamo visto con Git.

Commit changes

Add files via upload

Commit changes <-

3. Accendi il sito (GitHub Pages)

7Settings

In alto, nella fila di schede del repository, clic sull'**ultima a destra** : **Settings** (Impostazioni, con l'**ingranaggio**).

< > Code IssuesPull requestsActionsProjectsSecurity Settings <-

8Pages

Nel **menu a sinistra** scendi un po' e clic su **Pages**.

9Scegli main e salva

Sotto **Build and deployment** : **Source** = **Deploy from a branch**. In **Branch** clic su **None** e scegli **main** ; accanto lascia **/ (root)**. Clic su **Save** (Salva).

General

Collaborators

Branches

Actions

Pages

Build and deployment

Source: Deploy from a branch ▾

Branch: main ▾ / (root) ▾ Save

10Aspetta 1-2 minuti

Premi F5 ogni tanto. In alto compare **Your site is live at …** (Il tuo sito è online) con il bottone **Visit site**. Cliccalo: **FATTO! Il tuo sito è su internet.**

Your site is live at **https://tuonome.github.io/mio-sito/** Visit site

11Mostralo

L'indirizzo è https://TUONOME.github.io/mio-sito/ (TUONOME = il tuo nome utente GitHub). Aprilo sul **telefono** e fallo vedere a un compagno. L'hai fatto tu!

4. Se qualcosa non va

Problema	Soluzione
Vedo **404** / **There isn't a GitHub Pages site here**	Aspetta ancora 2 minuti e premi F5. Controlla che il file si chiami proprio **index.html** (minuscolo) e che in Pages ci sia **main** salvato.
Il sito si vede ma **senza colori**	Nel repository manca **style.css** , o ha un nome diverso. Ricaricalo con **Add file -> Upload files**.
Non trovo un bottone	Guarda il disegno: posizione e colore. Poi alza la mano: ti aiuto io.

Vuoi cambiare il sito dopo? Nel repository clic sul file (es. **style.css**) -> la **matita** in alto a destra -> cambia -> bottone verde **Commit changes**. Dopo un minuto, F5 sul sito: è aggiornato. Ogni modifica è una **nuova versione**.

Carta e penna: scrivi sul quaderno il tuo indirizzo **github.io** e lo schema: **PC** (cartella mio-sito) -> **upload + commit** -> **GitHub** (repository) -> **Pages** -> **sito pubblico**.

Classe 3 · Dispensa 2 · v1.0 · Se GitHub cambia l'aspetto delle pagine, i passi restano gli stessi.

v1.0

Compito — Il mio primo sito pubblico

Classe 3 · Informatica · 05/10/2026 · si consegna su Classroom · italiano

Cosa consegni: il **Documento** che trovi già pronto nel compito su Classroom (c'è già il tuo nome), con **5 parti separate**. HTML e CSS **NON** vanno insieme: ognuno nella sua parte.

1Apri il Documento

Su **Classroom** apri il compito **Il mio primo sito pubblico**. Clic sul Documento con il **tuo nome** : si apre.

2Parte 1 — Il link

Apri il tuo sito pubblicato, clic sulla **barra dell'indirizzo** in alto, Ctrl + C. Nel Documento, sotto **1. Il link** , Ctrl + V. Deve contenere **github.io**.

3Parte 2 — Tutto l'HTML

Clic **destro** su **index.html** -> **Apri con** -> **Blocco note**. Ctrl + A (seleziona tutto), Ctrl + C. Nel Documento, sotto **2. Codice HTML** , Ctrl + V.

4Parte 3 — Tutto il CSS, a parte

Stessa cosa con **style.css** : lo incolli sotto **3. Codice CSS**. **Non** mischiarlo con l'HTML.

5Parte 4 — Lo screenshot

Apri il sito **pubblicato** (quello con github.io). Premi Win + Shift + S e seleziona la finestra **con anche la barra dell'indirizzo**. Nel Documento, sotto **4. Screenshot** , Ctrl + V.

6Parte 5 — Spiegalo con parole tue

Rispondi con 1-2 frasi: a) a cosa serve il file style.css? b) cosa NON sono riuscito a fare e perché? c) cosa NON ho capito?

7Consegna

Torna sul compito in Classroom e clic sul bottone **blu Consegna** (a destra).

Controlla prima di consegnare

	Il link si apre e contiene github.io
	L'HTML è **tutto** : dalla prima riga <!DOCTYPE html> all'ultima </html>
	Il CSS è in una **parte separata**
	Lo screenshot mostra la **barra dell'indirizzo**
	Sul sito **niente cognome, telefono, email**
	Ho risposto alle 3 domande della parte 5

Come viene valutato (voto in decimi)

Cosa guardo	**Punti**
Il sito è pubblicato e il link funziona	**2**
HTML completo e corretto	**2**
CSS in un file separato, collegato: i colori si vedono sul sito	**2**
Il sito è tuo: testi e colori scelti da te	**1,5**
Screenshot con la barra dell'indirizzo	**1**
Parte 5: spieghi con parole tue	**1**
Privacy rispettata	**0,5**
Totale	**10**

Non hai finito in tempo? Consegna lo stesso quello che hai fatto e scrivilo nella parte 5: conta anche il pezzo fatto. Un errore non è una vergogna: succede a tutti i programmatori.

Classe 3 · Compito · v1.0

L07 07/10/2026 — Il sito migliorato: commit e History

Libro parziale: lezioni/2026-10-07_sito-migliorato.md

Che cosa abbiamo fatto

1. **Quando:** 07/10/2026, Laboratorio.
2. **Argomenti:** Compito 3 «Il sito migliorato» (HTML e CSS su GitHub, commit e History).

La teoria e l'esercitazione

v1.0

Dispensa 3 — Migliora il tuo sito direttamente su GitHub

Classe 3 · Informatica · 07/10/2026 · la matita, il commit, la storia delle versioni · italiano

Cosa ottieni oggi: nel tuo sito compaiono due **bottoni-link** che cambiano colore quando ci passi sopra, e la scheda ha un'**ombra**. Modifichi i file **direttamente su GitHub** , senza scaricare niente: ogni modifica è un **commit** , cioè una nuova **versione** salvata nella storia.

Il sito è pubblico: anche oggi solo nome o soprannome, niente dati personali.

1. Il codice nuovo

Il codice si copia dalla **pagina del corso** con il bottone **Copia** (link nel compito su Classroom).

RIQUADRO 3 — da aggiungere in index.html (prima di)

```
    <h2>I miei siti preferiti</h2>
    <p>
      <a class="bottone" href="https://it.wikipedia.org/wiki/HTML" target="_blank">Cos'è l'HTML</a>
      <a class="bottone" href="https://www.w3schools.com/css/" target="_blank">Imparo il CSS</a>
    </p>
```

RIQUADRO 4 — da aggiungere in fondo a style.css

```
.scheda {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.2);
}

.bottone {
  display: inline-block;
  background: darkblue;
  color: white;
  padding: 8px 14px;
  border-radius: 8px;
  text-decoration: none;
  margin: 4px;
}

.bottone:hover {
  background: orange;
}
```

2. I passi, uno alla volta

1Apri il tuo repository

Vai su github.com, entra (Sign in) e apri il repository **mio-sito** (in alto a sinistra, o dal tuo profilo).

2Apri index.html e premi la matita

Nell'elenco dei file clic su **index.html**. In alto a destra del codice clic sulla **matita** (Modifica).

github.com/tuonome/mio-sito/blob/main/index.html

mio-sito / index.html

Raw <- matita ⋮

3Incolla il RIQUADRO 3

Trova la riga

Lavorare con i computer.

(o il tuo sogno). Clic alla fine della riga, premi Invio e incolla il RIQUADRO 3 (Ctrl + V). Deve stare **prima** di .

4Salva: Commit changes

In alto a destra bottone **verde Commit changes…** -> si apre una finestrella -> di nuovo **Commit changes** (verde).

Cancel changes Commit changes… <- 1

Commit changes

Commit message: Update index.html

Commit changes <- 2

5Ora style.css

Torna al repository (clic su **mio-sito** in alto), clic su **style.css** -> **matita**. Vai **in fondo** , premi Invio e incolla il RIQUADRO 4. Poi **Commit changes…** -> **Commit changes**.

6Guarda il sito

Aspetta 1 minuto, apri il tuo sito (https://tuonome.github.io/mio-sito/) e premi Ctrl + F5. Passa con il mouse sui bottoni: diventano arancioni! **FATTO!**

7La storia delle versioni

Nel repository clic sul file **index.html** -> in alto a destra **History** (Cronologia, icona con l'orologio). Vedi la lista dei tuoi **commit** : ogni riga è una versione del file, con data e messaggio. Fai uno **screenshot** : serve per il compito.

8Fallo tuo

Cambia i link con **due siti utili che piacciono a te** (niente social, niente siti vietati) e i colori dei bottoni. Ogni volta: matita -> Commit.

3. Cosa fa ogni pezzo

Pezzo	Cosa fa
testo	un **link** : **href** è l'indirizzo dove porta
target="_blank"	apre il link in una scheda nuova
class="bottone"	il nome che usa il CSS per dare l'aspetto
.bottone:hover	lo stile quando il mouse **passa sopra**
box-shadow	l'ombra della scatola
commit	una versione salvata, con data e messaggio

Non vedi i cambiamenti? Aspetta ancora un minuto e premi Ctrl + F5 (ricarica forzata). Se il sito si rompe: apri **History** , guardi la versione di prima e la ricopi. Con Git **niente si perde**.

Carta e penna: disegna una linea del tempo con i tuoi commit (pallino = versione) e scrivi sotto cosa hai cambiato in ognuno.

Classe 3 · Dispensa 3 · v1.0

v1.0

Compito 3 — Il sito migliorato

Classe 3 · Informatica · 07/10/2026 · si consegna su Classroom · italiano

Cosa consegni: il **Documento** già pronto nel compito su Classroom (c'è già il tuo nome), una parte sotto l'altra.

Privacy: sul sito solo nome o soprannome. **Screenshot:** Win + Shift + S, poi nel Documento Ctrl + V.

1Parte 1

Il link del tuo sito

2Parte 2

Screenshot del sito con i due bottoni-link

3Parte 3

Il codice CSS che hai aggiunto (RIQUADRO 4, con i tuoi colori)

4Parte 4

Screenshot della History (la lista dei commit)

5Parte 5

Spiegalo con parole tue: a) a cosa serve href? b) cos'è un commit e perché con Git niente si perde?

6Riflessione e consegna

Scrivi cosa NON sei riuscito a fare e cosa NON hai capito. Poi in Classroom bottone **blu Consegna**.

Come viene valutato (voto in decimi)

Cosa guardo	Punti
Sito con i due bottoni funzionanti	**3**
CSS aggiunto e personalizzato	**2**
Screenshot della History	**2**
Spiegazione con parole tue	**2**
Riflessione: cosa non ho capito	**1**

L08 08/10/2026 — Il treno blindato: squadre e Pull Request

Libro parziale: lezioni/2026-10-08_treno-pull-request.md

Che cosa abbiamo fatto

1. **Quando:** 08/10/2026, Laboratorio.
2. **Argomenti:** Progetto del treno: squadre con ruoli (CEO, Motore, Personaggi, Scenari, Test qualità), Pull Request entrate nel gioco.

La teoria e l'esercitazione

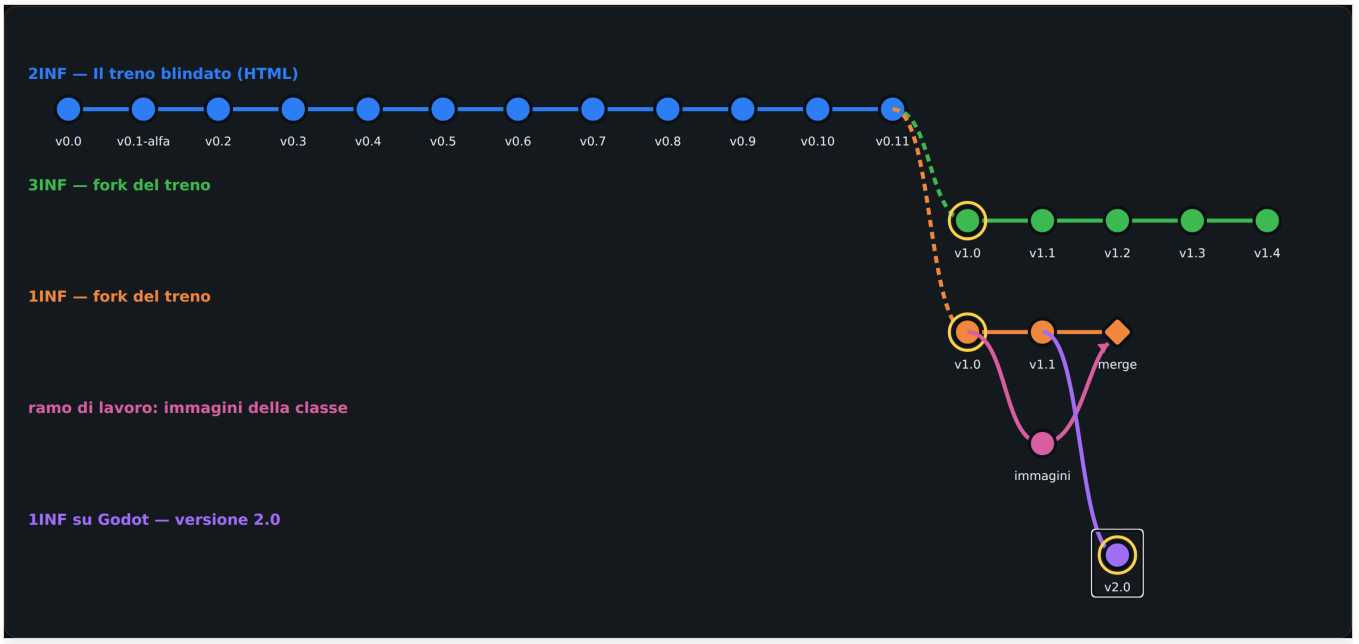
Testo per la tua AI: chiedile di spiegarti ogni parte nella tua lingua.

01 Che cosa facciamo oggi

1. Carta e penna sul banco: oggi si disegna il grafo a mano.
2. VINCI SUBITO (5 minuti). Apri il grafo, clicca il primo pallino BLU (v0.0) e premi il bottone verde «Apri questa versione»: gioca 30 secondi. Poi torna al grafo, clicca il pallino VIOLA (v2.0) e gioca ancora. Sul foglio scrivi 2 cose che sono cambiate.
3. Leggi le 8 parole del grafo (tabella qui sotto) e copiale sul foglio con la traduzione nella tua lingua.
4. Clicca i pallini del TUO ramo (arancione per la 1INF, blu per la 2INF): leggi «Cosa è cambiato e perché». Trova la versione dove c'è una cosa fatta dalla tua classe.
5. SCHEMA A MANO: disegna sul foglio un grafo con 3 colori: il ramo principale, un fork e un ramo di lavoro che rientra con un merge. Sotto ogni pallino scrivi il numero di versione.
6. PROVA DEL NOVE: spiega a voce al compagno, con parole tue, cos'è un merge e perché la versione Godot si chiama 2.0 e non 1.2.

Indirizzo del grafo (scrivilo esattamente così):

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/grafo/
```



Il grafo delle versioni

02 Le 8 parole del grafo

|—|—|—|—|—|

03 Perché

1. **Perché i numeri cambiano così.** v0.x = le prove, prima della versione stabile (la 2INF ha fatto 12 prove: v0.0 ⟶ v0.11). 1.0 = la prima versione «vera» di un ramo (il fork della 1INF e della 3INF). 1.1, 1.2 = versioni MINOR: piccole aggiunte (un cattivo, un cielo). 2.0 = versione MAJOR: cambia una cosa grossa (il motore Godot), il primo numero sale e il secondo riparte da 0.

2. **Perché si fanno i rami.** Per provare una cosa nuova SENZA rompere il gioco che funziona. Il prof ha fatto la funzione «immagini della classe» su un ramo a parte; quando funzionava l'ha unita (merge) al ramo della 1INF. Nelle aziende si lavora sempre così: ognuno sul suo ramo, poi Pull Request, controllo e merge.

3. **Perché Godot (v2.0).** Godot è un motore vero per videogiochi, gratis e portabile (non si installa), usato anche nel lavoro. I vostri cattivi ci sono ancora: stanno nei file di dati (cattivi.json), uguali a prima. Ponte da Lazarus: Lazarus REAGISCE (aspetta un clic), Godot PULSA (_process gira da solo circa 60 volte al secondo).

04 La storia, ramo per ramo

1.1 **v0.0 — Prima idea: il finestrino laterale.** Il treno visto dal finestrino di lato: paesaggio che scorre, cattivi che sbucano da dietro alberi e case, vetro da 5 colpi. Poi la classe ha disegnato alla lavagna la vista dalla cabina. 1.2 **v0.1-alfa — Demo del prof sul progetto della lavagna.** Vista dalla cabina lungo i binari (progetto della classe), cattivi da finestre, alberi e barili, facce della classe e copertina disegnata con l'IA, tachimetro, installabile come app. Da qui la classe corregge il tiro. 1.3 **v0.2 — I personaggi della squadra PERSONAGGI.** Quattro personaggi disegnati con l'IA (tre agenti e il Boss dei cattivi), bottone «I personaggi», le loro facce nel gioco. Prima modifica arrivata dalla classe. 1.4 **v0.3 — Più lento: richiesta della squadra TEST QUALITÀ.** Prima richiesta di modifica consegnata su Classroom dalla Triade: «il gioco va troppo veloce». Velocità del treno da 1 a 0,7. 1.5 **v0.4 — Audit degli investitori: personaggi nascosti che escono.** Il cattivo è un personaggio intero nascosto dietro gli oggetti che esce di lato, più grande; treno al minimo (0,5); tolte 2 foto senza faccia. 1.6 **v0.5 — Le regole della squadra MOTORE FISICA (Pull Request #104).** Prima Pull Request vera (#104) del capo MOTORE FISICA: cattivi sparano dopo 3 secondi, 100 punti per cattivo, velocità 0,5. Corretto 0,5 ⟶ 0.5 (in JSON i decimali vogliono il punto). 1.7 **v0.6 — Il bosco bruciato della squadra SCENARI.** Richiesta SCENARI: cielo rosso cupo, alberi bruciati e rottami fumanti vicino

ai binari. Richiesta PERSONAGGI: il cattivo dice «Hahahahaha, perdente!» quando spara. 1.8 **v0.7 — Omini interi che escono dal nascondiglio.** Indicazione del prof: omini interi (testa, busto, braccia, gambe, arma) al posto dei cerchietti; si alzano da dietro il nascondiglio ed escono di lato con un «!»; quando li colpisci cadono. 1.9 **v0.8 — Mimetica, facce senza cerchio e il fucile d'assalto.** Indicazioni del prof: facce senza cerchio (solo la faccia), barretta del tempo sopra la testa, omini in mimetica, l'Agente Speciale con il fucile d'assalto. 1.10 **v0.9 — Teste in proporzione.** Indicazione del prof: la testa degli omini lontani era troppo grossa (misura minima fissa); ora è sempre in proporzione al corpo. 1.11 **v0.10 — I migliori punteggi.** Indicazione del prof: classifica dei primi 10 con soprannome a fine partita, mostrata nella schermata iniziale; record della classe da record-classe.json. 1.12 **v0.11 — I punteggi si salvano sempre.** Difetto segnalato dal prof: il punteggio ora si salva da solo a fine partita, il soprannome si conferma anche con Invio; il gioco chiede sempre alla rete la versione nuova.

2.1 **v1.0 — Il fork della 3INF: traversine di legno e nuvole.** La 3INF fa la sua copia (fork) del gioco della 2INF (v0.11). Traversine di legno con venature e nodi, nuvole nel cielo. 2.2 **v1.1 — Black Panter (PR #1 e #2, SCENARI).** Pull Request #1 (PC 18) e #2 (PC 17) dalla consegna su Classroom: il cattivo Black Panter con la mimetica nera e argento, esce da una porta, dice «Yahuu!». 2.3 **v1.2 — Montagna, tempo che peggiora, Travor.** PR #4 Dani e #5 Ale (SCENARI): si parte dalla montagna, cielo sereno, il tempo peggiora a ogni livello. PR #6 Tomas (PERSONAGGI): il cattivo Travor, mimetica grigia. 2.4 **v1.3 — Cespugli, greggi, armi diverse, nuovi cattivi.** PR #8 Mirko: cespugli e armi diverse; #13 Dani e #14 Ale: greggi in montagna; #10 Ariel D. e #11 Tomas: Travor e il protagonista Nash; #12 Eric: Franciscos. 2.5 **v1.4 — Treno più veloce, Nash, Dylan, cielo che cambia subito.** PR #17 Noah: velocità 2; #15 Ariel D.: Nash; #19 Emmanuel: Dylan; #18 Eric: Franciscos azzurro e bianco; #20 Dani e #21 Ale: difetto del cielo sistemato.

3.1 **v1.0 — Il fork della 1INF.** Copia (fork) del gioco della 2INF alla versione v0.11. Da qui partono le Pull Request della 1INF. 3.2 **v1.1 — I primi cattivi e i cieli della 1INF.** Pull Request accettate: cattivi Hidra (Giexther A.), EI (Yinlin L.), Mister Bar Berry (Matteo A.); cielo arancione in montagna (Giexther A.), cielo di guerra nel bosco (Yinlin L.), cielo viola in città (Wesley J.).

4.1 **immagini — Nuova funzione su un ramo separato.** Il prof prova una cosa nuova su un RAMO a parte, così il gioco che funziona non si rompe: cartelle per le immagini (cattivi, paesaggi, difetti, revisioni), galleria con i nomi, foto prese da sole dalle consegne di Classroom. Quando funziona: MERGE (unione) nel ramo della 1INF.

5.1 **v2.0 — Il treno su Godot.** Stesso gioco della 1INF v1.1 (cattivi Hidra, EI, Mister Bar Berry e i cieli della 1INF), ma rifatto con il motore Godot 4.7 ed esportato per il browser. È cambiato il motore, cioè una cosa grossa: per questo il numero passa da 1.x a 2.0 (versione major).

L09 12/10/2026 — Versioning e Agile

Libro parziale: lezioni/2026-10-12_versioning-agile.md

Che cosa abbiamo fatto

1. **Quando:** 12/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** Il Documento come ramo dell'albero delle versioni; come si chiede una modifica (Pull Request PR1, PR2…); Manifesto Agile e 10 regole; backlog Agile.

La teoria e l'esercitazione

Testo per la tua AI: chiedile di spiegarti ogni parola nella tua lingua.

01 I link della lezione

1. Il grafo delle versioni (clicca i pallini)

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/grafo/
```

1. Tutte le versioni del vostro treno, giocabili

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-blindato/versioni/
```

1. Il changelog del treno (la lista dei cambiamenti)

```
https://github.com/nicolaregge-pulse/corso-informatica/blob/main/docs/giochi/treno-blindato/CHANGELOG.md
```

1. La storia dei commit del ramo treno-2inf

```
https://github.com/nicolaregge-pulse/corso-informatica/commits/treno-2inf
```

1. Il grafo vero di Git su GitHub (Network)

```
https://github.com/nicolaregge-pulse/corso-informatica/network
```

1. Il fork della 3INF: il loro treno

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-3inf/
```

1. La versione 2.0 su Godot

```
https://nicolaregge-pulse.github.io/corso-informatica/giochi/treno-godot/
```

02 Le 10 parole, con il nostro gioco

1. **Versione.** Una «fotografia» del progetto in un momento preciso, con un numero. Il vostro treno è passato da v0.0 (prima idea, finestrino laterale) a v0.11 (i punteggi si salvano sempre): 12 versioni in un giorno.

1.2 Prova: Apri «Tutte le versioni» e gioca la v0.0 e la v0.11: cosa è cambiato?

1. **Git e GitHub.** Git è il programma che ricorda TUTTA la storia del progetto: chi ha cambiato cosa, quando e perché. GitHub è il sito dove il progetto (il repository) sta online. Il nostro repository si chiama corso-informatica.

2.2 Prova: Apri «La storia dei commit»: ogni riga è un salvataggio.

1. **Commit.** Un salvataggio nella storia di Git, con un messaggio che spiega il cambiamento. Esempio vero di ieri: 7724900 «Treno blindato v0.11: i punteggi si salvano sempre». Il numero strano (7724900) è il codice del commit.

3.2 Prova: Trova nella storia il commit della v0.5 (la Pull Request #104).

1. **Branch (ramo).** Una linea di lavoro separata. Si apre un ramo per provare una cosa nuova senza rompere il gioco che funziona. Il vostro gioco sta nel ramo treno-2inf; il prof ha provato le «immagini della classe» su un ramo a parte.

4.2 Prova: Nel grafo: quante linee (rami) vedi?

1. **Fork (copia).** La copia di un progetto intero che cresce per conto suo. Ieri la 3INF ha fatto il fork del vostro treno alla v0.11 e lo ha portato alla v1.4 (montagna, Black Panter, Nash…); anche la 1INF.

5.2 Prova: Nel grafo: da quale vostra versione parte la linea verde della 3INF?

1. **Push.** Mandare i propri commit dal PC al repository online (GitHub). Ieri, dopo ogni versione, il prof ha fatto il push: per questo il gioco sul sito cambiava da solo.

6.2 Prova: Perché il gioco sul telefono cambiava senza scaricare niente?

1. **Pull.** Il contrario del push: prendere da GitHub l'ultima versione e portarla sul proprio PC. La v0.11 del treno «chiede sempre alla rete la versione nuova»: è un pull automatico.

7.2 Prova: Quando fai un pull, cosa ricevi?

1. **Pull Request (proposta di modifica).** «Ho fatto una modifica: la vuoi nel progetto?». Qualcuno la controlla e, se va bene, fa il MERGE (la unisce). Ieri le vostre richieste su Classroom erano Pull Request: la #104 della squadra MOTORE FISICA è diventata la v0.5.

8.2 Prova: Chi decide se una Pull Request entra nel gioco?

1. **Changelog (registro dei cambiamenti).** La lista, versione per versione, di cosa è cambiato e perché. Il treno ha il suo CHANGELOG.md: chi gioca sa cosa c'è di nuovo, chi programma sa dove cercare un errore.

9.2 Prova: Apri il changelog: qual è la novità della v0.7?

1. **Numeri delle versioni (major.minor).** v0.x = prove; 1.0 = prima versione stabile (il fork della 3INF parte da 1.0); 1.1, 1.2 = piccole aggiunte (minor); 2.0 = cambio grosso (major): il gioco rifatto con Godot è la v2.0.

10.2 Prova: Perché la versione Godot non si chiama v1.5?

L10 14/10/2026 — Verifica sul versioning e il merge

Libro parziale: lezioni/2026-10-14_verifica-versioning-merge.md

Che cosa abbiamo fatto

1. **Quando:** 14/10/2026, Laboratorio.
2. **Argomenti:** Verifica sul versioning (fork, Pull Request, merge, changelog); revisione del backlog Agile con le richieste al gioco del treno.

La teoria e l'esercitazione

Le schede della lezione

1. Le versioni e il changelog · ১. সংস্করণ ও changelog

Una **versione** è un salvataggio del gioco con un **numero** e una riga che dice **cosa cambia**. Il numero ha due parti: **major.minor**. Da v1.4 a v1.5 cambia il numero dopo il punto: è un passo piccolo, **minor**. Da v1.5 a v2.0 cambia il primo numero: è un cambio grande, **major** (per esempio un motore nuovo). Il **changelog** è il registro di tutte le modifiche, versione per versione: che cosa è cambiato e da quale richiesta. Un programma **cambia** perché chi lo usa chiede cose nuove e perché si trovano **difetti** da sistemare: nella v1.4 Dani e Ale hanno trovato che il cielo cambiava in ritardo, ed è stato sistemato.

সংস্করণ মানে খেলার একটি সেভ: একটি **নম্বর** আর কী বদলেছে তার একটি লাইন। নম্বরের দুই অংশ: **major.minor**। v1.4 থেকে v1.5: ছোট পরিবর্তন (**minor**)। v1.5 থেকে v2.0: বড় পরিবর্তন (**major**)। **changelog** হলো সব পরিবর্তনের তালিকা। প্রোগ্রাম বদলায়, কারণ মানুষ নতুন জিনিস চায় আর ভুল ঠিক করতে হয়।

2. Fork, ramo e merge · ২. fork, শাখা ও merge

L'08/10 la 3INF ha fatto il suo **fork**: una **copia** del treno della 2INF (versione v0.11), per lavorarci senza toccare il gioco della 2INF. Da lì è partita la v1.0 della 3INF. Un **ramo** (branch) è una strada di lavoro parallela nello stesso progetto. Il **merge** unisce una modifica (o un ramo) dentro il gioco: lo fa il prof quando accetta una Pull Request.

০৮/১০ তারিখে 3INF **fork** করেছে: 2INF-এর ট্রেন খেলার একটি **কপি** (v0.11), যাতে ওদের খেলা না বদলায়। **শাখা** (branch) মানে একই প্রজেক্টে পাশাপাশি আরেকটি কাজের রাস্তা। **merge** মানে একটি পরিবর্তন খেলার সাথে যোগ করা: শিক্ষক Pull Request গ্রহণ করলে এটা করেন।

3. La Pull Request · ৩. Pull Request

Una **Pull Request** (PR) è una **richiesta di modifica scritta**. Ha quattro parti: **Titolo**, **Cosa cambia**, **Dove** (in quale punto del gioco), **Perché**. L'08/10 le squadre SCENARI, PERSONAGGI, MOTORE FISICA e TEST QUALITÀ hanno mandato le PR dalla #1 alla #21: da quelle sono nate le versioni dalla v1.1 alla v1.4. Lunedì 12/10 sono arrivate 14 richieste: da queste nascono oggi le versioni nuove, dalla v1.5 in poi.

Pull Request (PR) মানে লিখিত একটি পরিবর্তনের অনুরোধ। এর চারটি অংশ: **শিরোনাম**, **কী বদলাবে**, **কোথায়**, **কেন**। ০৮/১০-এ দলগুলো PR #1 থেকে #21 পাঠিয়েছে: তা থেকে v1.1 থেকে v1.4 হয়েছে। ১২/১০-এ ১৪টি অনুরোধ এসেছে: আজ তা থেকে v1.5 আর পরের সংস্করণ হচ্ছে।

4. Il backlog e i pesi · ৪. backlog ও ওজন

Il **backlog** è la coda dei lavori: tutte le richieste, in ordine. Ogni richiesta ha un **peso**: quanto lavoro serve sul codice. I pesi sono i numeri di **Fibonacci**: 1, 2, 3, 5, 8 (ogni numero è la somma dei due prima). 1 = un colore, 2 = un numero nelle regole, 3 = un personaggio, 5 = un ambiente, 8 = due cose insieme o una regola nuova. Si fanno prima le richieste più leggere; a parità di peso, chi è arrivato prima.

backlog মানে কাজের সারি: সব অনুরোধ, ক্রম অনুযায়ী। প্রতিটি অনুরোধের একটি **ওজন** আছে: কোডে কত কাজ লাগবে। ওজন হলো Fibonacci সংখ্যা: ১, ২, ৩, ৫, ৮ (প্রতিটি আগের দুইটির যোগফল)। ১ = একটি রং, ২ = একটি সংখ্যা, ৩ = একটি চরিত্র, ৫ = একটি পরিবেশ, ৮ = দুইটি জিনিস একসাথে। আগে হালকা কাজ হয়।

5. Agile: sprint e retrospettiva · ৫. Agile: sprint ও retrospettiva

Agile è un modo di lavorare delle aziende di software. Il **Manifesto Agile** dice, per esempio: un programma che **funziona** conta più di tanti documenti; **rispondere ai cambiamenti** conta più che seguire un piano fisso; le **persone** e il parlarsi contano più degli strumenti. Si lavora a **sprint**: giri di lavoro brevi, con un obiettivo, alla fine dei quali esce una versione che funziona. Alla fine dello sprint c'è la **retrospettiva**: la squadra dice che cosa è andato bene e che cosa migliorare.

Agile হলো সফটওয়্যার কোম্পানির কাজের একটি পদ্ধতি। **Agile Manifesto** বলে: চালু প্রোগ্রাম অনেক কাগজের চেয়ে বেশি জরুরি; পরিবর্তনে সাড়া দেওয়া নির্দিষ্ট পরিকল্পনার চেয়ে বেশি জরুরি; মানুষ আর কথা বলা যন্ত্রের চেয়ে বেশি জরুরি। কাজ হয় **sprint**-এ: ছোট কাজের পালা, একটি লক্ষ্য, শেষে একটি চালু সংস্করণ। শেষে **retrospettiva**: দল বলে কী ভালো হয়েছে আর কী ভালো করা যায়।

6. Le regole della squadra · ৬. দলের নিয়ম

Il gioco è di **tutta la classe**. Regola: **aggiungere sì, cancellare no**. Il cattivo Travor l'ha chiesto Tomas: se hai un'idea, aggiungi un personaggio tuo, non cancelli quello di un compagno. Il gioco è **pubblico**: lo vede chiunque su Internet. Per questo **niente foto di persone vere** (privacy): lunedì 3 richieste sono state respinte per questo motivo. Si possono rimandare con un personaggio disegnato o di un videogioco.

খেলাটি **পুরো ক্লাসের**। নিয়ম: **যোগ করা যায়, মুছে ফেলা যায় না**। Travor চরিত্রটি Tomas চেয়েছিল: তোমার নতুন ধারণা থাকলে নিজের চরিত্র যোগ করো, বন্ধুরটা মুছে ফেলো না। খেলাটি ইন্টারনেটে **সবাই দেখতে পায়**। তাই **আসল মানুষের ছবি নয়** (গোপনীয়তা): সোমবার এই কারণে ৩টি অনুরোধ বাতিল হয়েছে। আঁকা চরিত্র বা ভিডিও গেমের চরিত্র দিয়ে আবার পাঠানো যায়।

L11 15/10/2026 — Sei il Product Manager: un team Agile

Libro parziale: lezioni/2026-10-15_product-manager-agile.md

Che cosa abbiamo fatto

1. **Quando:** 15/10/2026, 2 ore, Laboratorio.
2. **Argomenti:** Il gioco del Product Manager (due clienti, sprint, priorità, backlog), le cerimonie Agile e una verifica.

La teoria e l'esercitazione

Le schede della lezione

1. Lo sprint e le cerimonie

Lo **sprint** è l'unità di tempo dell'Agile (in azienda una settimana, lun-ven; nel gioco 15 minuti). Ha una **capacità** fissa: il team fa al massimo **8 punti** di lavoro. Ogni sprint gira in 4 **cerimonie**: **Planning** (si sceglie), **Daily** (ogni giorno, cosa ho fatto/faccio/mi blocca), **Review** (si mostra la versione ai clienti), **Retrospettiva** (il team migliora il suo modo di lavorare).

2. Backlog, peso e rischio

Il **backlog** è la lista ordinata delle richieste. Ogni richiesta ha un **peso** (quanto lavoro serve: 1, 2, 3, 5, 8) e un **rischio** sul software (quanto può rompere le altre parti: 1, 2, 3). Il **valore** dice quanto serve ai clienti.

3. Il planning: mettere insieme i pezzetti

Il **planning** è **mettere insieme i pezzetti** nell'ordine giusto: non si fa tutto e non a caso, si scelgono le richieste che danno il risultato più grande dentro gli 8 punti. Il filo è la **priorità = valore ÷ (peso + rischio)**: prima quelle che valgono tanto e costano poco. I clienti non pesano uguale: **TrenoCity conta 3**, la **Ludoteca conta 1**.

একটি Agile দলের মতো কাজ করা

3INF · 15/10/2026 · la teoria di oggi: Product Manager, backlog, sprint, planning, release

Il giocoIl cruscottoIl monitor

In questa lezione:

1. Perché «Agile»
2. Chi fa che cosa: i ruoli
3. Il backlog: la lista delle richieste
4. Peso, rischio e valore
5. I clienti non pesano uguale
6. La priorità: la formula
7. Lo sprint

8. Le quattro cerimonie
9. Il planning: mettere insieme i pezzetti
10. La release e il changelog
11. Esempio sul nostro treno
12. Glossario
13. Saperlo spiegare

1. Perché «Agile»

Quando un gruppo di persone costruisce un programma, il rischio è lavorare tanto nella direzione sbagliata e accorgersene troppo tardi. Il metodo Agile risponde così: si lavora a **piccoli passi**, si consegna spesso qualcosa che funziona, si ascolta chi userà il programma e si corregge subito. Meglio sbagliare presto su una cosa piccola che tardi su una grande.

যখন একদল মানুষ একটি প্রোগ্রাম বানায়, বিপদ হলো অনেক কাজ ভুল দিকে করা এবং দেরিতে বোঝা। **Agile** পদ্ধতি: ছোট ছোট ধাপে কাজ, ঘন ঘন কাজ করে এমন কিছু জমা দেওয়া, ব্যবহারকারীর কথা শোনা, এবং সঙ্গে সঙ্গে ঠিক করা। বড় ভুল দেরিতে করার চেয়ে ছোট ভুল আগে করা ভালো।

Nel nostro corso lo abbiamo vissuto: il treno della classe è cresciuto una versione alla volta, con le vostre richieste. Questo è Agile.

2. Chi fa che cosa: i ruoli

In un vero team di sviluppo ognuno ha un compito preciso.

Ruolo	Che cosa fa
Product Manager প্রোডাক্ট ম্যানেজার	Parla con i clienti, capisce che cosa vale di più, decide l'ordine del backlog e che cosa entra nello sprint. Sa dire anche dei «no».
Sviluppatori ডেভেলপার	Stimano il peso (quanto lavoro serve) e scrivono il codice.
Tester টেস্টার	Provano la versione prima della consegna e trovano i bug.
Clienti গ্রাহক	Pagano il lavoro e dicono che cosa serve a loro. Non tutti pesano uguale.

প্রোডাক্ট ম্যানেজার: গ্রাহকদের সাথে কথা বলে, কোনটা বেশি মূল্যবান বোঝে, backlog-এর ক্রম ঠিক করে, «না»-ও বলে। ডেভেলপার: ওজন অনুমান করে ও কোড লেখে। টেস্টার: জমা দেওয়ার আগে পরীক্ষা করে। গ্রাহক: টাকা দেয় ও কী দরকার বলে; সবাই সমান ওজনের নয়।

3. Il backlog: la lista delle richieste

Il backlog è la lista ordinata di tutte le richieste di modifica. Ogni richiesta entra in coda e aspetta il suo turno. Non si fa tutto subito: si fa prima ciò che conta di più.

Backlog হলো সব পরিবর্তনের অনুরোধের সাজানো তালিকা। প্রতিটি অনুরোধ সারিতে অপেক্ষা করে। সব একসাথে নয়: আগে যা বেশি গুরুত্বপূর্ণ।

4. Peso, rischio e valore

Ogni richiesta del backlog ha tre numeri, e servono per decidere:

1. Peso (sforzo): quanto lavoro serve a farla. Si usano i numeri 1, 2, 3, 5, 8: più è alto, più lavoro.
2. Rischio sul software: quanto quella modifica può rompere le altre parti del programma. 1 basso, 2 medio, 3 alto.
3. Valore: quanto quella richiesta serve ai clienti.

প্রতিটি অনুরোধের তিনটি সংখ্যা: **ওজন** (কত কাজ: ১,২,৩,৫,৮), **সফটওয়্যারে ঝুঁকি** (অন্য অংশ ভাঙার সম্ভাবনা: ১ কম, ৩ বেশি), **মূল্য** (গ্রাহকের কাছে কত দরকার)।

5. I clienti non pesano uguale

Nel nostro gioco c'erano due clienti:

TrenoCity — cliente grande, **conta × 3**. Una catena di sale giochi: vuole un gioco stabile e veloce, senza bug.

Ludoteca Arcobaleno — cliente piccolo, **conta × 1**. Un centro per ragazzi: vuole un gioco bello e divertente.

Accontentare il cliente grande vale di più. Il valore totale di una richiesta si calcola così:

Valore = (valore per TrenoCity × 3) + (valore per Ludoteca × 1)

দুই গ্রাহক: TrenoCity (বড়, × ৩, স্থির ও দ্রুত গেম চায়) এবং Ludoteca (ছোট, × ১, সুন্দর ও মজার গেম চায়)। মোট মূল্য = (TrenoCity-এর মূল্য × ৩) + (Ludoteca-এর মূল্য × ১)।

6. La priorità: la formula

Il Product Manager mette in cima le richieste che danno tanto valore e costano poco lavoro. La regola è:

Priorità = Valore ÷ (Peso + Rischio)

Più la priorità è alta, prima si fa. Due richieste con lo stesso valore: si fa prima quella che pesa e rischia meno.

অগ্রাধিকার = মূল্য ÷ (ওজন + ঝুঁকি)। অগ্রাধিকার বেশি হলে আগে করা হয়। একই মূল্যের দুটি অনুরোধে, যেটির ওজন ও ঝুঁকি কম সেটি আগে।

7. Lo sprint

Lo sprint è l'**unità di tempo** del metodo Agile. In un'azienda di solito dura **una settimana, da lunedì a venerdì**; nel nostro gioco durava 15 minuti. Ha una capacità fissa: il team riesce a fare al massimo **8 punti** di lavoro per sprint. Alla fine di ogni sprint si consegna una **versione nuova** del programma.

Sprint হলো Agile-এর সময়ের একক। কোম্পানিতে সাধারণত এক সপ্তাহ (সোম–শুক্র); আমাদের খেলায় ১৫ মিনিট। ধারণক্ষমতা: প্রতি sprint-এ সর্বোচ্চ ৮ পয়েন্ট কাজ। প্রতিটি sprint শেষে একটি নতুন সংস্করণ জমা হয়।

8. Le quattro cerimonie

Ogni sprint gira in un cerchio di quattro incontri fissi del team, poi ricomincia:

1. Planninglunedì
2. Dailyogni giorno
3. Reviewvenerdì
4. Retrovenerdì
5. Sprint planning (lunedì): il Product Manager porta il backlog in ordine, il team sceglie che cosa fare nella settimana.
6. Daily (ogni giorno, 15 minuti in piedi): ognuno dice che cosa ha fatto ieri, che cosa fa oggi e che cosa lo blocca.
7. Sprint review (venerdì): si mostra la versione nuova ai clienti e si ascolta il loro parere.
8. Retrospettiva (venerdì): il team, da solo, decide come lavorare meglio nello sprint dopo.

চারটি অনুষ্ঠান: ১) Planning (সোমবার): কী করা হবে বেছে নেওয়া। ২) Daily (প্রতিদিন): গতকাল কী, আজ কী, কী আটকাচ্ছে। ৩) Review (শুক্রবার): গ্রাহকদের নতুন সংস্করণ দেখানো। ৪) Retrospective (শুক্রবার): দল ঠিক করে কীভাবে আরও ভালো কাজ করবে।

9. Il planning: mettere insieme i pezzetti

Oggi, dal vostro giro di esperienze, è uscito il concetto più forte: il planning è **mettere insieme i pezzetti**. Vuol dire scegliere e ordinare le richieste in modo che, dentro gli 8 punti dello sprint, si ottenga il **risultato più grande** per i clienti. Non si fa tutto e non si fa a caso: il filo che tiene insieme i pezzetti è la priorità (valore ÷ peso + rischio).

Essere Product Manager vuol dire proprio questo: davanti a tante richieste, mettere insieme i pezzi giusti, nel momento giusto.

Planning = টুকরোগুলো একসাথে জোড়া দেওয়া: ৮ পয়েন্টের মধ্যে গ্রাহকদের জন্য সবচেয়ে বড় ফল পেতে অনুরোধ বেছে ও সাজানো। সব নয়, এলোমেলোও নয়: সূত্র হলো অগ্রাধিকার।

10. La release e il changelog

Alla fine dello sprint si consegna una release: una versione con un numero fisso (v3.0, v3.1…), pronta per i clienti. Insieme si scrive il changelog, l'elenco di tutte le modifiche fatte in quella versione, con le **note di rilascio** scritte per chi gioca. Il changelog è il registro tecnico; le note di rilascio sono il racconto per l'utente.

Release: নির্দিষ্ট নম্বরসহ একটি সংস্করণ (v3.0…), গ্রাহকের জন্য প্রস্তুত। **Changelog**: সেই সংস্করণের সব পরিবর্তনের তালিকা; রিলিজ নোট হলো ব্যবহারকারীর জন্য লেখা।

11. Esempio sul nostro treno

Immagina questo backlog per il treno:

Richiesta	Peso	Rischio	TrenoCity ×3	Ludoteca ×1	Valore	Priorità
Correggere il bug dei nemici	3	1	5	3	18	**4,5**
Schermata GAME OVER	2	1	2	4	10	**3,3**
Nuovo motore della fisica	8	3	4	2	14	**1,3**

Il bug ha priorità 4,5: si fa per primo (vale tanto, costa poco). Il motore nuovo ha priorità 1,3: vale, ma pesa 8 e rischia 3, quindi va dopo. Con 8 punti puoi prendere il bug (3) + GAME OVER (2) + qualcos'altro di leggero: resti dentro la capacità e accontenti i due clienti.

উদাহরণ: বাগ ঠিক করার অগ্রাধিকার ৪.৫ (বেশি মূল্য, কম খরচ) — আগে। নতুন ফিজিক্স ইঞ্জিন ১.৩ (ওজন ৮, ঝুঁকি ৩) — পরে। ৮ পয়েন্টে বাগ (৩) + GAME OVER (২) + হালকা কিছু নেওয়া যায়।

12. Glossario

Agile	metodo di lavoro a piccoli passi, con consegne frequenti.
Backlog	la lista ordinata delle richieste.
Peso	quanto lavoro serve per una richiesta.
Rischio	quanto una modifica può rompere il resto.
Valore	quanto una richiesta serve ai clienti.
Priorità	valore ÷ (peso + rischio): dice cosa fare prima.
Sprint	unità di tempo, con capacità fissa (8 punti).
Release	la versione consegnata ai clienti.
Changelog	l'elenco delle modifiche di ogni versione.
Product Manager	chi decide l'ordine del backlog e cosa entra nello sprint.

13. Saperlo spiegare

Se sai rispondere a queste domande con parole tue, hai capito davvero:

1. Che cos'è il backlog e perché le richieste hanno un peso?
2. Che differenza c'è tra peso e rischio?
3. Come si calcola la priorità? Fai un esempio.
4. Perché TrenoCity conta più della Ludoteca?
5. Che cosa vuol dire «mettere insieme i pezzetti» nel planning?
6. Quali sono le quattro cerimonie e a cosa serve ognuna?

নিজের ভাষায় এই প্রশ্নগুলোর উত্তর দিতে পারলে তুমি সত্যিই বুঝেছ: backlog কী ও ওজন কেন; ওজন ও ঝুঁকির পার্থক্য; অগ্রাধিকার কীভাবে হিসাব হয়; TrenoCity কেন বেশি গোনে; planning-এ «টুকরো জোড়া» মানে কী; চারটি অনুষ্ঠান কী কী।

Prof. Nicola Regge · Laboratorio di informatica

L12 19/10/2026 — Il sito diventa portfolio

Libro parziale: lezioni/2026-10-19_portfolio.md

Che cosa abbiamo fatto

1. **Quando:** 19/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** Ripasso di HTML5, CSS3 e GitHub Pages; la pagina competenze.html «Cosa so fare» con menu e link nella Home; verifica e recupero.

La teoria e l’esercitazione

Le schede della lezione

1. Dove siamo: il tuo sito · ১. আমরা কোথায় আছি

Hai già un **sito pubblico** su Internet: lo hai fatto tu, dall'8 ottobre. Sta in un **repository** su GitHub (quasi tutti lo hanno chiamato **mio-sito**) e ha dentro almeno due file: **index.html** (la pagina) e **style.css** (i colori e lo stile). Chi ha fatto la Dispensa 4 ha anche una **seconda pagina** e un **menu** per passare da una pagina all'altra. L'indirizzo è **https://tuonome.github.io/mio-sito/**. Oggi il sito diventa il tuo **portfolio**: una pagina che dice **cosa sai fare**, da mostrare a un'azienda per lo stage.

তোমার একটি ওয়েবসাইট আছে, GitHub-এ **mio-sito** নামে। আজ এটি তোমার **portfolio** হবে: কী কী পারো, কোম্পানিকে দেখানোর জন্য।

2. Ripasso HTML: i mattoni della pagina · ২. HTML

HTML è la lingua delle pagine web. Ogni pezzo sta tra due **tag**: uno apre `<p>`, uno chiude `</p>`. Quelli che usi tu:
`<h1>` titolo grande · `<h2>` titolo piccolo · `<p>` paragrafo (un pezzo di testo) · `<ul>` elenco, con dentro un `<li>` per ogni riga · `<a href="pagina.html">Testo</a>` link, cioè un collegamento a un'altra pagina · `<div class="scheda">` una scatola che raggruppa.
La pagina ha una **testa** `<head>` (il titolo della scheda del browser e il collegamento al CSS) e un **corpo** `<body>` (tutto quello che si vede).

HTML-এ প্রতিটি অংশ দুটি **tag**-এর মাঝে থাকে: `<p>` খোলে, `</p>` বন্ধ করে। h1 বড় শিরোনাম, ul/li তালিকা, a href লিংক।

3. Ripasso CSS: i colori e lo stile · ৩. CSS

Il **CSS** decide com'è fatta la pagina: colori, bordi, spazi. Sta nel file **style.css**, collegato alla pagina con la riga `<link rel="stylesheet" href="style.css">` dentro head. Una regola CSS è fatta così:

```
.menu {   background: darkblue;   padding: 10px; }
```

.menu è il **selettore**: dice *a chi* si applica (qui: a tutto quello che ha `class="menu"`). Tra le graffe ci sono le **proprietà**: `background` il colore di sfondo, `padding` lo spazio interno, `color` il colore del testo, `border-radius` gli angoli arrotondati. Ogni proprietà finisce con **;**. Il punto davanti (`.menu`) vuol dire «classe».

CSS পাতার রং আর স্টাইল ঠিক করে, ফাইল **style.css**-এ। **.menu** হলো selector (কাকে প্রযোজ্য), গোল বন্ধনীর ভিতরে property: background, padding, color।

4. Ripasso GitHub Pages: come si pubblica · ৪. GitHub Pages

Il sito vive nel **repository** su github.com. Per cambiare un file: apri il file, clicca la **matita** (in alto a destra), scrivi, poi **Commit changes** due volte. Il **commit** è il salvataggio con la data e un messaggio: GitHub tiene tutta la storia. Per un file nuovo: **Add file ⟶ Create new file**, scrivi il nome, incolla, commit.
Dopo il commit, **GitHub Pages** ricostruisce il sito: aspetta **1 minuto**, poi nel sito premi **Ctrl+F5** (ricarica senza memoria). Se vedi **404**, il nome del file non è quello che c'è scritto nel link: controlla maiuscole e minuscole, **competenze.html** tutto minuscolo.

GitHub-এ ফাইল বদলাতে: ফাইল খোলো, **pencil** চাপো, লেখো, **Commit changes** দুইবার। ১ মিনিট পর সাইটে **Ctrl+F5**। **404** মানে ফাইলের নাম ভুল।

5. Il portfolio: a cosa serve, e la regola d’oro · ৫. Portfolio

Un **portfolio** è la pagina che racconta **cosa sai fare**: le competenze **tecniche** (HTML, CSS, GitHub, Lazarus, montare un PC…) e quelle **personali** (chiedo aiuto quando mi blocco, se sbaglio riprovo, aiuto un compagno). È il tuo **biglietto da visita** per lo stage e per un colloquio: un'azienda lo apre e in 30 secondi capisce chi sei.
Regola d'oro (privacy): la pagina è **pubblica**, la vede chiunque. Scrivi **solo cose vere e solo cose positive**, e solo il tuo **nome o soprannome**: niente cognome, telefono, indirizzo, classe, scuola, foto di altri. Le cose personali (difficoltà, famiglia, da dove vieni) **non** vanno sul sito: le scrivi nella verifica, che legge solo il prof.

Portfolio = কী পারো তার পাতা: technical আর personal দক্ষতা। স্টেজ আর ইন্টারভিউর জন্য। **নিয়ম:** পাতা সবাই দেখে, তাই শুধু সত্য আর ভালো কথা, শুধু নাম বা ডাকনাম; ফোন, ঠিকানা, পরিবারের কথা নয়।

6. Il codice: la pagina competenze.html · ৬. কোড

Questa è la pagina intera. Oggi la copi **da qui** dentro GitHub (il bottone in alto a destra del riquadro, oppure seleziona tutto e Ctrl+C). Poi la cambi con le **tue** cose vere.

```
<!DOCTYPE html> <html lang="it"> <head>   <meta charset="utf-8">   <meta name="viewport" content="width=dev
```

Guarda com'è fatta: il **menu** in alto (nav) con due link, un **h1**, una **scheda** (div) con quattro **h2** e gli elenchi. La riga `<link rel="stylesheet" href="style.css">` le dà gli stessi colori della tua Home.
এই পুরো পাতাটি কপি করে GitHub-এ বসাও, তারপর নিজের সত্য কথা দিয়ে বদলাও।

7. Il menu nella Home e i suoi colori · ৭. মেনু

Perché dalla Home si arrivi al portfolio, anche **index.html** deve avere il menu. Si mette **subito dopo** la riga `<body>`:

```
  <nav class="menu">     <a href="index.html">Home</a>     <a href="competenze.html">Cosa so fare</a>   </n
```

Se hai già il menu della Dispensa 4, aggiungi solo la riga di **competenze.html**. Se hai anche **passione.html**, lascia anche quella.
Se il menu resta senza colori, in fondo a **style.css** ci va questo:

```
.menu {   background: darkblue;   padding: 10px;   border-radius: 10px; } .menu a {   color: white;   text-
```

index.html-এও মেনু লাগবে, `<body>`-এর ঠিক পরে। মেনুতে রং না থাকলে **style.css**-এর শেষে এই কোড।

8. LA PRATICA, passi 1-3: la pagina nuova su GitHub · ৮. অনুশীলন ১-৩

Un passo alla volta. Ogni passo è una cosa sola.
1) Vai su **github.com**, entra, apri il tuo repository **mio-sito**.
2) Clicca il bottone grigio **Add file** (in alto a destra, vicino al verde), poi **Create new file**. Nella casella del nome scrivi esattamente **competenze.html** (tutto minuscolo).
3) Nel riquadro grande incolla la pagina della scheda 6 (Ctrl+V). Poi il bottone verde **Commit changes...**, poi di nuovo **Commit changes**. **FATTO!** Il file esiste.
Rispondi alla domanda qui sotto: così il monitor sa che sei a questo punto.
১) github.com ⟶ **mio-sito**। ২) **Add file** ⟶ **Create new file**, নাম **competenze.html**। ৩) কোড পেস্ট, **Commit changes** দুইবার।

9. LA PRATICA, passi 4-6: fallo tuo e collega la Home · ৯. অনুশীলন ৪-৬

4) **Fallo tuo, solo cose vere.** Apri **competenze.html**, clicca la **matita**. Cambia gli elenchi: lascia solo le competenze che hai davvero, aggiungine di tue (una cosa che sai fare al PC, in Lazarus, con le mani...). Scrivi la frase «Qui ho imparato a ...» e la **tua frase per un'azienda**. Commit.
5) Apri **index.html**, matita. Subito dopo la riga `<body>` incolla il menu della scheda 7 (se c'è già, aggiungi solo la riga di competenze.html). Commit.
6) Aspetta 1 minuto. Apri il tuo sito, **Ctrl+F5**. Clicca **Cosa so fare**, poi **Home**. **FATTO!** Il tuo portfolio è online. Copia l'indirizzo della pagina dalla barra del browser: ti serve per la verifica.
৪) **competenze.html** ⟶ pencil ⟶ নিজের সত্য দক্ষতা লেখো ⟶ Commit। ৫) **index.html**-এ `<body>`-এর পরে মেনু ⟶ Commit। ৬) ১ মিনিট পর সাইট খোলো, **Ctrl+F5**। ঠিকানাটি কপি করো।

10. Se qualcosa non va, e la sfida per chi ha finito · ১০. সমস্যা ও চ্যালেঞ্জ

Vedo 404 cliccando «Cosa so fare»: il file deve chiamarsi esattamente **competenze.html**, minuscolo, come nel menu. **Il menu è senza colori**: in fondo a style.css incolla il CSS della scheda 7, aspetta un minuto, Ctrl+F5. **La scheda bianca non ha il bordo**: ti manca il CSS della Dispensa 3 (.scheda), lo trovi nella pagina della classe. **Non vedo le modifiche**: hai fatto il commit? Hai aspettato un minuto? Ctrl+F5?
Mostralo: leggi al compagno la tua frase per un'azienda, come a un colloquio, 30 secondi.
Sfida per chi ha finito: 1) una terza pagina **passione.html** (la cosa che ti piace di più) con la sua riga nel menu; 2)

cambia un colore del menu e lo sfondo della scheda; 3) nella Home metti un link al portfolio anche in fondo alla pagina. Ti blocchi? Scrivi la domanda all'assistente, in fondo alla pagina: ti risponde.

404: ফাইলের নাম **competenze.html** ছোট হাতের। মেনুতে রং নেই: style.css-এর শেষে CSS। শেষ করলে: **passione.html** তৃতীয় পাতা, রং বদলাও। আটকে গেলে নিচে সহকারীকে প্রশ্ন করো।

11. Ultimi 10 minuti: prepariamo mercoledì (lavoro in coppia) · ১১. বুধবারের প্রস্তুতি

Mercoledì lavorate **in coppia**: il compagno scrive una pagina nel **tuo** sito e tu nel suo, come in un'azienda. Per farlo, oggi vi date il permesso a vicenda. Il prof vi dice chi è il vostro compagno.
1) Chiedi al compagno il suo **nome utente GitHub** (quello dell'indirizzo, prima di .github.io).
2) Nel tuo repository **mio-sito** clicca **Settings** (l'ingranaggio, in alto a destra nella barra del repository).
3) Nel menu a sinistra clicca **Collaborators** (Collaboratori). Se chiede la password, rimettila.
4) Clicca il bottone verde **Add people**, scrivi il nome utente del compagno, sceglilo dall'elenco e conferma.
5) Il compagno riceve l'invito: lo trova nella **campanella** in alto a destra su github.com (o nella posta) e preme **Accept invitation**. Poi fate lo stesso al contrario. **FATTO!** Mercoledì potete lavorare uno nel sito dell'altro.
Rispondi alla domanda qui sotto.

বুধবার জোড়ায় কাজ: সঙ্গী তোমার সাইটে একটি পাতা লিখবে। আজ: **Settings ⟶ Collaborators ⟶ Add people**, সঙ্গীর GitHub নাম লেখো। সঙ্গী ঘণ্টার আইকনে **Accept invitation** চাপে। তারপর উল্টোটা।

12. Per chi ha finito: una FOTO DI SFONDO al tuo sito · ১২. পটভূমির ছবি

Solo se hai la bandiera (portfolio completo). Il sito diventa tuo davvero con una foto di sfondo.
1) Scegli una foto: una TUA (fatta da te, senza persone riconoscibili) oppure una libera da pixabay.com. Scaricala sul PC e rinominala **sfondo.jpg** (tutto minuscolo).
2) Nel tuo repository: **Add file ⟶ Upload files**, trascina **sfondo.jpg**, poi il verde **Commit changes**.
3) Apri **style.css** ⟶ matita. Dentro le graffe di **body { }** aggiungi queste tre righe, poi **Commit changes**:

```
background-image: url("sfondo.jpg"); background-size: cover; background-attachment: fixed;
```

4) Se il testo si legge male: alle regole **.scheda** (o **main**) aggiungi **background: #fff; padding: 16px; border-radius: 12px;** così il testo sta su un riquadro bianco.
5) Aspetta 1 minuto, apri il sito, **Ctrl+F5**: la foto è dietro a tutte le pagine (anche «Cosa so fare», perché usano lo stesso style.css). **FATTO!** Rispondi alla domanda qui sotto.

শুধু থাকলে। একটি ছবি (নিজের বা pixabay.com থেকে), নাম **sfondo.jpg**। Repository-তে **Add file ⟶ Upload files**। তারপর **style.css**-এর **body { }**-তে: background-image: url("sfondo.jpg"); background-size: cover; background-attachment: fixed; ⟶ Commit। ১ মিনিট পরে Ctrl+F5।

v1.0

Dispensa 4 — La seconda pagina e il menu

Classe 3 · Informatica · 08/10/2026 · un sito vero ha più pagine collegate · italiano

Cosa ottieni oggi: il tuo sito ha **due pagine** (Home e La mia passione) e un **menu** in alto per passare dall'una all'altra, come i siti veri. Le due pagine usano lo **stesso style.css** : cambi un colore e cambia in tutto il sito.

1. Il codice nuovo

RIQUADRO 5 — file nuovo passione.html (la seconda pagina)

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>La mia passione</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav class="menu">
    <a href="index.html">Home</a>
    <a href="passione.html">La mia passione</a>
  </nav>
```

```
  <h1>La mia passione: il calcio</h1>

  <div class="scheda">
    <h2>Perché mi piace</h2>
    <p>Mi piace giocare in squadra e allenarmi con gli amici.</p>

    <h2>Tre cose che so fare</h2>
    <ol>
      <li>Passare la palla con precisione</li>
      <li>Correre veloce</li>
      <li>Parare un rigore</li>
    </ol>

    <h2>Lo sapevi che...</h2>
    <p>Il primo mondiale di calcio si è giocato nel 1930.</p>
  </div>

  <p class="piede">Sito creato da me - 2026</p>
</body>
</html>
```

RIQUADRO 6 — il menu: da mettere in index.html, subito dopo

```
  <nav class="menu">
    <a href="index.html">Home</a>
    <a href="passione.html">La mia passione</a>
  </nav>
```

RIQUADRO 7 — in fondo a style.css

```
.menu {
  background: darkblue;
  padding: 10px;
  border-radius: 10px;
}

.menu a {
  color: white;
  text-decoration: none;
  font-weight: bold;
  margin: 0 12px;
}

.menu a:hover {
  color: orange;
}
```

2. I passi, uno alla volta

1Crea il file nuovo

Nel repository **mio-sito** : bottone **Add file** (grigio) -> **Create new file** (Crea nuovo file). In alto, nel campo del nome, scrivi passione.html.

github.com/tuonome/mio-sito/new/main

mio-sito / passione.html

Enter file contents here

2Incolla la seconda pagina

Nel riquadro grande incolla il RIQUADRO 5. Poi **Commit changes...** -> **Commit changes**.

3Fallo tuo

Con la **matita** cambia la pagina: la **tua** passione (sport, musica, cucina, un videogioco, un paese...), perché ti piace, tre cose che sai fare, una curiosità. Poi Commit.

4Il menu anche nella Home

Apri **index.html** -> **matita**. Subito dopo la riga incolla il RIQUADRO 6. Commit.

5Lo stile del menu

Apri **style.css** -> **matita** -> in fondo incolla il RIQUADRO 7. Commit.

6Prova il menu

Aspetta 1 minuto, apri il sito, Ctrl + F5. Clic su **La mia passione** , poi su **Home** : passi da una pagina all'altra. **FATTO! Hai un sito a due pagine.**

7Mostralo

L'indirizzo della seconda pagina è https://tuonome.github.io/mio-sito/passione.html. Aprila sul telefono e falla vedere a un compagno.

3. Se qualcosa non va

Problema	Soluzione
Clic su **La mia passione** e vedo **404**	Il file deve chiamarsi esattamente **passione.html** (minuscolo), come nel menu.
La seconda pagina è **senza colori**	Manca la riga <link rel="stylesheet" href="style.css"> nel <head>.
Il menu c'è ma è **senza stile**	Il RIQUADRO 7 non è in style.css, oppure aspetta un minuto e Ctrl + F5.

Carta e penna: disegna la **mappa del sito** : due rettangoli (index.html e passione.html) con le frecce del menu, e una freccia da ciascuno verso style.css.

Classe 3 · Dispensa 4 · v1.0

L13 21/10/2026 — Struttura del sito, CSS e lavoro in coppia

Libro parziale: lezioni/2026-10-21_grafica-pull-request.md

Che cosa abbiamo fatto

1. **Quando:** 21/10/2026, 2 ore, Laboratorio.
2. **Argomenti:** Albero del sito, nomi giusti, cartella img, immagine di sfondo, animazione con transition e @keyframes, numero di versione; invito come collaboratore, ramo, Pull Request e merge sul sito del compagno; quiz su Moduli.

La teoria e l'esercitazione

Le schede della lezione

1. Prima di tutto: il sito in ordine

> Un sito funziona solo se i file sono **nel posto giusto** e con il **nome giusto**. Nella cartella principale del repository servono:
> `index.html` · `style.css` · `competenze.html` · `passione.html` · la cartella `img` con le tue immagini.
>
> Apri L'albero del mio sito, scrivi l'indirizzo del tuo sito e premi **Controlla**. Sistema le croci rosse **una alla volta** finché è tutto verde. Questo è il primo traguardo di oggi.

2. Le tre regole dei nomi

> 1) **Tutto minuscolo**: GitHub distingue `Index.html` da `index.html`: per lui sono due file diversi, e il link non funziona.
> 2) **Niente spazi**: `mia foto.jpg` no, `mia-foto.jpg` sì.
> 3) **Il collegamento uguale al nome, lettera per lettera**: se il file è `sfondo.jpeg`, nel CSS scrivi `sfondo.jpeg`, non `sfondo.jpg`.
>
> Per rinominare su GitHub: apri il file, matita, cambia il nome in alto, **Commit changes**. Per spostarlo nella cartella img scrivi `img/` davanti al nome.

3. Un'immagine di sfondo

Carica la tua immagine: **Add file ⟶ Upload files**, poi rinominala in `img/sfondo.jpg`. Poi in `style.css`, dentro `body { }`, aggiungi:

```
background-image: url("img/sfondo.jpg"); background-size: cover; background-attachment: fixed;
```

Se il testo non si legge bene, metti il contenuto in un riquadro semitrasparente:

```
main { background: rgba(255,255,255,0.85); padding: 20px; border-radius: 12px; }
```

L'immagine la scegli tu: è il tuo sito.

4. Una piccola animazione

Transition: un cambiamento morbido quando passi sopra con il mouse.

```
nav a { transition: transform 0.3s; } nav a:hover { transform: scale(1.15); }
```

@keyframes: un'animazione che parte da sola, per esempio il titolo che entra piano.

```
@keyframes entra {   from { opacity: 0; transform: translateY(-20px); }   to   { opacity: 1; transform: tr
```

Poche animazioni e leggere: un sito che si muove troppo stanca chi lo guarda.

5. A coppie: ramo, Pull Request, merge

Ora lavori sul sito del compagno, come in un'azienda.
1) Il compagno ti aggiunge come **collaboratore**: Settings ⟶ Collaborators ⟶ Add people.
2) Tu, nel suo repository, apri un file, matita, e in basso scegli **Create a new branch** (un ramo: la tua copia per provare). Aggiungi una sezione «Il mio compagno di lavoro»: tre righe su di lui e un consiglio per il suo sito.
3) Premi **Propose changes** e poi **Create pull request**.
4) Il compagno apre la Pull Request, guarda le modifiche e preme **Merge pull request**. Il suo sito ora ha la tua sezione!
Poi vi scambiate i ruoli.

6. Il numero di versione

Nel piè di pagina di `index.html` scrivi la versione: `<footer>Versione 1.0</footer>`.
Ogni Pull Request accettata che aggiunge qualcosa fa salire il secondo numero: **1.0 ⟶ 1.1 ⟶ 1.2**. Il primo numero sale solo per un cambiamento grande (un sito tutto nuovo: 2.0).
Sfida finale: tutti e due cambiate la **stessa riga** del titolo in due rami diversi. GitHub dice **conflitto**: scegliete insieme quale tenere (Resolve conflicts), poi merge. Avete risolto il vostro primo conflitto, come i programmatori veri.

L14 22/10/2026 — Recupero: il sito di 3 pagine

Libro parziale: lezioni/2026-10-22_recupero-sito.md

Che cosa abbiamo fatto

1. **Quando:** 22/10/2026, Laboratorio.
2. **Argomenti:** Recupero: un sito di 3 pagine con sfondo, animazione e versione.

La teoria e l'esercitazione

Le schede della lezione

1. Oggi: il tuo sito funziona, per tutti

Oggi ognuno arriva ad avere il suo **sito personale** che funziona: 3 pagine con il menu, una foto di sfondo, un'animazione e il numero di versione.
Si lavora **solo su github.com**, nel tuo repository del sito. Niente programmi da aprire.
Guarda la tabella in alto: per ognuno dice **che cosa ti manca**. Fai **solo** i passi che ti mancano. Quello che c'è già e

funziona non lo tocchi.
Il sito ha questi file, tutti nella cartella principale (la foto nella cartella **img**):

```
index.html        la Home style.css         i colori, lo sfondo, l'animazione competenze.html   che cosa so
```

Come si mette un file su GitHub (sempre uguale):
• Il file **non c'è**: nel repository clicca **Add file ⟶ Create new file**, scrivi il nome, incolla il testo, poi **Commit changes… ⟶ Commit changes**.
• Il file **c'è già** ma la tabella dice di rifarlo: aprilo, clicca la **matita**, **Ctrl+A**, incolla il testo nuovo, poi **Commit changes… ⟶ Commit changes**.
Il testo lo copi da **La pagina dei file pronti** (bottone verde in alto): lì ogni file ha il suo bottone **Copia**.

2. PASSO 1: la Home (index.html)

La Home è la prima pagina che si apre. Si chiama sempre **index.html**, tutto minuscolo.
1) Copia il file **index.html** dalla pagina dei file pronti (bottone verde in alto).
2) Mettilo nel repository come spiegato nella scheda 1.
3) **Fallo tuo**: al posto di **IL TUO NOME** scrivi il tuo nome, e completa le frasi con le tue parole.

FATTO! Nel repository c'è index.html con il tuo nome.

3. PASSO 2: lo stile (style.css)

Il file **style.css** dà i colori, la foto di sfondo e l'animazione a tutte le pagine.
1) Copia **style.css** dalla pagina dei file pronti e mettilo nel repository.
2) **Fallo tuo**: cambia il colore del menu: la riga `background: #1f6feb;` dentro `nav`. Prova per esempio `#e11d48` (rosso), `#16a34a` (verde), `#9333ea` (viola).
Dentro ci sono già lo **sfondo** (`background-image`) e l'**animazione** (`@keyframes`): il riquadro bianco entra piano dal basso.

FATTO! Nel repository c'è style.css.

4. PASSO 3 e 4: le pagine competenze e passione

Le altre due pagine si chiamano esattamente **competenze.html** e **passione.html** (tutto minuscolo, senza spazi). Se il nome è diverso, il menu non le trova.
1) Copia **competenze.html** dai file pronti, mettila nel repository, scrivi 3 cose che sai fare.
2) Copia **passione.html**, mettila nel repository, scrivi la tua passione (sport, musica, giochi, cucina…).
Hai già una pagina con un nome sbagliato (comp.html, passioni.html)? Crea quella con il nome giusto e copia dentro il tuo testo.

FATTO! Nel repository ci sono competenze.html e passione.html.

5. PASSO 5: la foto di sfondo nella cartella img

1) Cerca una foto che ti piace (Google Immagini, oppure una tua). Salvala sul PC con il nome:

```
sfondo.jpg
```

Attenzione: deve finire con **.jpg**. Se Windows la salva come .jfif o .webp, nella finestra di salvataggio scrivi il nome **sfondo.jpg** per intero.
2) Nel repository crea la cartella img: **Add file ⟶ Create new file**, come nome scrivi

```
img/LEGGIMI.md
```

e nel testo scrivi una parola qualsiasi, poi **Commit changes**. La barra / crea la cartella.
3) Clicca la cartella **img**, poi **Add file ⟶ Upload files**, trascina dentro **sfondo.jpg** e clicca **Commit changes**.

FATTO! Nel repository c'è img/sfondo.jpg.

6. PASSO 6: guarda il tuo sito su internet

1) Nel repository clicca **Settings** (in alto a destra, con l'ingranaggio) ⟶ a sinistra **Pages**.
2) In **Branch** deve esserci **main** e **/ (root)**. Se c'è scritto None, scegli **main** e clicca **Save**.
3) In alto nella stessa pagina compare l'indirizzo del sito, così:

```
https://tuonome.github.io/nome-del-repository/
```

4) Aprilo. Le modifiche arrivano dopo 1 o 2 minuti: premi **Ctrl+F5** per ricaricare.
5) Clicca il menu: Home, Competenze, Passione. Si aprono tutte?

FATTO! Il tuo sito si apre su internet, con la foto di sfondo e il menu che funziona. Fallo vedere al compagno!

7. Se qualcosa non va (succede a tutti, anche ai pro)

1) **Pagina bianca o errore 404** quando clicco il menu: il nome del file è sbagliato. Deve essere esattamente competenze.html o passione.html.
2) **Non vedo lo sfondo**: la foto non si chiama sfondo.jpg, oppure non è nella cartella img. Guarda la tabella in alto.
3) **Il sito non cambia**: aspetta 2 minuti e premi Ctrl+F5.
4) **Il sito è tutto senza colori**: manca style.css, oppure si chiama in un altro modo (Style.css, stile.css).
5) **I miei file sono in una sottocartella** (Mio-sito/...): il sito li cerca nella cartella principale. Dillo all'assistente: ti aiuta a spostarli.

FATTO! Hai risolto da solo o con l'assistente.

L15 26/10/2026 — Object Pascal e Java

Libro parziale: lezioni/2026-10-26_pascal-java.md

Che cosa abbiamo fatto

1. **Quando:** 26/10/2026, 3 ore, Laboratorio.
2. **Argomenti:** Confronto tra Object Pascal e Java: Hello World, compilato e interpretato.

La teoria e l'esercitazione

Perché con Object Pascal (Lazarus) si impara prima: meno codice, variabili senza «new», finestre disegnate con il mouse

Classe 3 · 26/10/2026 · Versione 1.1

Lo stesso programma scritto in due linguaggi. Guarda quante righe servono e quante parole nuove devi imparare: capirai perché per cominciare usiamo Lazarus.

01 Due linguaggi

Object Pascal è il linguaggio di **Lazarus** (e di Delphi). **Java** è un linguaggio molto usato nelle aziende e nelle app Android.

	Object Pascal (Lazarus)	Java
Come diventa un programma	compilato: diventa un file .exe	compilato in «bytecode», poi interpretato dalla Java Virtual Machine
Che cosa serve sul PC	niente: il .exe parte da solo	Java installato (la macchina virtuale)
Le finestre	le disegni con il mouse	le scrivi con il codice

02 Hello World

Object Pascal

```
begin
  writeln('Hello World');
end.
```

Java

```
public class Ciao {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

In Java già per scrivere una frase servono una **classe** e il metodo **main**, con parole come **public**, **static**, **void**, **String[] args**. In Object Pascal bastano **begin** ed **end**.

03 Le variabili: niente «new»

In Object Pascal le variabili si **dichiarano** nella sezione **var** con nome e tipo, e poi si usano subito. Anche un array è già pronto.

Object Pascal

```
var
  nome: string;
  eta: integer;
  voti: array[1..5] of integer;
begin
  nome := 'Ali';
  eta := 16;
  voti[1] := 8;
  readln(nome);        // leggo dalla tastiera
end.
```

In Java i tipi semplici (int, double) vanno bene, ma per un array o per leggere dalla tastiera bisogna **creare un oggetto** con la parola **new**.

Java

```
String nome = "Ali";
int eta = 16;
int[] voti = new int[5];          // serve new
voti[0] = 8;                       // si parte da 0
Scanner tastiera = new Scanner(System.in);   // serve new
nome = tastiera.nextLine();
```

> ***Da sapere.*** *In Java l'array parte da 0; in Pascal decidi tu dove parte (qui da 1). In Java ogni cosa «grande» va creata con new prima di usarla.*

04 Una finestra con un bottone

Con Lazarus trascini il bottone sul Form, fai doppio clic e scrivi **una riga**:

Object Pascal (Lazarus)

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  ShowMessage('Hello World');
end;
```

In Java (con Swing) la finestra e il bottone si scrivono tutti con il codice, e ognuno si crea con **new**:

Java

```
JFrame finestra = new JFrame("Ciao");
JButton bottone = new JButton("Premi");
bottone.addActionListener(e ->
    JOptionPane.showMessageDialog(null, "Hello World"));
finestra.add(bottone);
finestra.setSize(300, 200);
finestra.setVisible(true);
```

05 Riassunto

Che cosa	Object Pascal	Java
Hello World	1 riga dentro begin...end	5 righe con classe e main
Variabili e array	nella sezione var, pronte subito	gli array e molti oggetti con new
Leggere dalla tastiera	readln	new Scanner(System.in)
Finestre e bottoni	disegnati con il mouse	scritti con il codice e new
Programma finito	un .exe che parte ovunque	serve Java installato

06 Per essere onesti

Anche in Object Pascal le **classi** più complesse si creano (**Create**) e poi si liberano (**Free**). Java invece libera la memoria da solo. E Java è usatissimo nel lavoro: per questo è bene sapere che esiste.

Object Pascal

```
var lista: TStringList;
begin
  lista := TStringList.Create;   // anche qui si crea
  lista.Add('Ali');
  lista.Free;                    // e si libera
end;
```

> ***Da sapere.*** *Per imparare a programmare e vedere subito il risultato, però, Object Pascal con Lazarus è la strada più semplice: meno parole da imparare, finestre con il mouse, un .exe che funziona ovunque.*

07 I due programmi completi, con la finestra e il bottone

Lazarus. Tutto questo codice lo scrive Lazarus da solo quando trascini il bottone sul Form. Tu scrivi **una riga** sola: quella con ShowMessage.

Object Pascal: Unit1.pas

```
unit Unit1;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls;

type
  TForm1 = class(TForm)
    Button1: TButton;
    procedure Button1Click(Sender: TObject);
  end;

var
  Form1: TForm1;

implementation

{$R *.lfm}
```

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  ShowMessage('Hello World');   // l'UNICA riga scritta da te
end;

end.
```

Java. Qui ogni riga la scrivi tu: la finestra, il bottone, che cosa succede al clic, la grandezza, la chiusura. Ogni oggetto si crea con **new**.

Java: Ciao.java

```
import javax.swing.*;

public class Ciao {
    public static void main(String[] args) {
        JFrame finestra = new JFrame("Ciao");
        JButton bottone = new JButton("Premi");
        bottone.addActionListener(e ->
            JOptionPane.showMessageDialog(finestra, "Hello World"));
        finestra.add(bottone);
        finestra.setSize(300, 200);
        finestra.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        finestra.setLocationRelativeTo(null);
        finestra.setVisible(true);
    }
}
```

08 Provare Java senza installare niente

A scuola non si installa niente. Ci sono due modi per vedere la finestra di Java.

1. **Nel browser, con JavaFiddle.** Apri l'indirizzo qui sotto, cancella il codice di esempio, incolla il programma Ciao.java e premi «Run»: la finestra con il bottone si apre dentro la pagina. Funziona solo se il firewall della scuola lascia passare il sito.
2. **Java portabile, come Godot.** Si scarica Java in versione .zip (per esempio «Eclipse Temurin JDK», file zip) e si scompatta in una cartella, per esempio C:\java\jdk: niente installazione e niente amministratore. Dalla Java 11 in poi basta un comando per far partire il file Ciao.java, senza compilarlo a parte.

Indirizzo di JavaFiddle, da scrivere così: **javafiddle.leaningtech.com**

Comando per Java portabile (cartella del file Ciao.java)

```
C:\java\jdk\bin\java.exe Ciao.java
```

> ***Da sapere.*** *Con Lazarus invece il programma diventa un file .exe: lo copi su una chiavetta e parte su qualunque PC Windows, anche senza Lazarus.*

09 Prova tu

1. Quante righe servono per Hello World in Object Pascal e quante in Java?
2. In Java, che cosa serve per creare un array di 5 numeri?
3. Perché un programma fatto con Lazarus parte su ogni PC Windows senza installare niente?

Soluzioni

1. Object Pascal: 1 riga (dentro begin…end). Java: 5 righe con classe e main.
2. La parola new: int[] voti = new int[5];
3. Perché è compilato in un file .exe; Java invece ha bisogno della sua macchina virtuale installata.

L16 28/10/2026 — Hello World in Lazarus e in Java

Libro parziale: lezioni/2026-10-28_hello-lazarus-java.md

Che cosa abbiamo fatto

1. **Quando:** 28/10/2026, 2 ore, Laboratorio.
2. **Argomenti:** Java senza installare (siti web), Eclipse portabile, HelloWorld con finestra e bottone, confronto Java e Lazarus.

La teoria e l'esercitazione

Le schede della lezione

1. Oggi: Hello World in due linguaggi · আজ: দুই ভাষায় Hello World

Questa pagina cambia durante la lezione: ogni tanto premi F5 per vedere la versione nuova.
এই পাতাটি ক্লাসের সময় বদলায়: মাঝে মাঝে F5 চাপো, নতুন সংস্করণ দেখতে।

Il mio cruscotto: voto, che cosa ho fatto e che cosa manca

Guardalo ogni tanto: ti dice a che punto sei e che cosa ti manca per consegnare.
মাঝে মাঝে দেখো: তুমি কোথায় আছো আর জমা দিতে কী বাকি আছে, তা বলে।

Lunedì avete visto la teoria: **Object Pascal** (Lazarus) e **Java** a confronto. Oggi li **provate davvero**, tutti e due.
সোমবার তোমরা তত্ত্ব দেখেছো: Object Pascal (Lazarus) আর Java তুলনা। আজ দুটোই সত্যি সত্যি চালিয়ে দেখবে।

Hello World («Ciao mondo») è il primo programma di ogni programmatore, in ogni linguaggio: serve a controllare che tutto funzioni.
Hello World হলো প্রত্যেক প্রোগ্রামারের প্রথম প্রোগ্রাম, যেকোনো ভাষায়: সব ঠিক চলছে কিনা দেখার জন্য।

Parte	Che cosa fai	Schede
A. Lazarus	un progetto pulito e una finestra che saluta	2, 3, 4, 5, 6
B. Java	Hello World in Java, nel browser, senza installare niente	7, 8
C. Consegna	il Documento su Classroom, entro le 14:00	9

A: Lazarus-এ একটি জানালা যা শুভেচ্ছা জানায়। B: ব্রাউজারে Java, কিছু ইনস্টল না করে। C: Classroom-এ ডকুমেন্ট জমা, ১৪:০০-এর মধ্যে।

Per aprire Lazarus: doppio clic sull'icona di Lazarus. Poi vai alla scheda 2.
Lazarus খুলতে: Lazarus আইকনে দুবার ক্লিক করো। তারপর ২ নম্বর কার্ডে যাও।

2. Partenza pulita: progetto nuovo in Documenti\Lazarus\Saluto · পরিষ্কার শুরু: নতুন প্রজেক্ট

Come si crea un progetto nuovo (la guida con le figure)

La regola è la stessa per tutti: nella cartella **Documenti ⟶ Lazarus** stanno tutti i tuoi progetti, e **ogni progetto ha la sua cartella**. Oggi il progetto si chiama `Saluto`.
নিয়ম সবার জন্য এক: Documenti ⟶ Lazarus ফোল্ডারে তোমার সব প্রজেক্ট থাকে, আর প্রতিটি প্রজেক্টের নিজের ফোল্ডার। আজকের প্রজেক্টের নাম Saluto।

```
Documenti\   Lazarus\          <- TUTTI i miei progetti     Saluto\          <- UN progetto = UNA
```

1. In alto a sinistra in Lazarus clicca il menu **File**, poi **Chiudi tutto**. Se chiede di salvare un progetto vecchio, clicca **No**.
উপরে বাঁদিকে File মেনুতে ক্লিক করো, তারপর «Chiudi tutto»। পুরনো প্রজেক্ট সেভ করতে বললে «No» চাপো।

2. In alto clicca il menu **Progetto**, poi **Nuovo progetto...**.
উপরে Progetto মেনুতে ক্লিক করো, তারপর «Nuovo progetto...»।

3. Nell'elenco a sinistra clicca **Applicazione**, poi in basso **OK**. Al centro compare la finestra vuota **Form1**.
বাঁদিকের তালিকায় «Applicazione» ক্লিক করো, তারপর নিচে OK। মাঝখানে খালি Form1 জানালা আসবে।

4. Subito si salva: menu **Progetto ⟶ Salva progetto con nome...**.
এখনই সেভ করো: Progetto মেনু ⟶ «Salva progetto con nome...»।

5. Nel pannello a sinistra clicca **Documenti**, poi fai doppio clic sulla cartella **Lazarus**.
বাঁদিকের প্যানেলে Documenti ক্লিক করো, তারপর Lazarus ফোল্ডারে দুবার ক্লিক করো।

6. In alto clicca **Nuova cartella**, scrivi questo nome e premi `Invio`:
উপরে «Nuova cartella» ক্লিক করো, এই নামটি লেখো আর Invio চাপো:

```
Saluto
```

7. Fai doppio clic sulla cartella `Saluto`. In alto ora c'è scritto **Documenti › Lazarus › Saluto**.
Saluto ফোল্ডারে দুবার ক্লিক করো। উপরে এখন লেখা থাকবে Documenti › Lazarus › Saluto।

8. Nella casella **Nome file** scrivi questo nome, poi clicca **Salva**:
Nome file ঘরে এই নামটি লেখো, তারপর Salva ক্লিক করো:

```
Saluto.lpi
```

9. Lazarus chiede il nome della unit (il file del codice): lascia `unit1.pas` e clicca **Salva**.
Lazarus ইউনিটের (কোডের ফাইল) নাম চাইবে: unit1.pas রেখে দাও আর Salva চাপো।

FATTO! In alto, nella barra del titolo di Lazarus, c'è scritto `Saluto` e il percorso che finisce con `Documents\Lazarus\Saluto`.
হয়ে গেছে! Lazarus-এর উপরের শিরোনামে Saluto আর Documents\Lazarus\Saluto লেখা আছে।

3. La finestra: TEdit, TButton, TLabel · জানালা: TEdit, TButton, TLabel

Tre pezzi dalla palette, linguetta **Standard**. Li conoscete già: oggi li usate insieme.
প্যালেটের Standard ট্যাব থেকে তিনটি অংশ। তোমরা এগুলো আগেই চেনো: আজ একসাথে ব্যবহার করবে।

Pezzo	A che cosa serve	Proprietà che conta
TEdit	chi usa il programma **scrive** il nome	`Text`
TButton	si **preme** per far succedere una cosa	`Caption`
TLabel	un testo da **leggere**: qui compare il saluto	`Caption`

TEdit: নাম লেখার ঘর। TButton: চাপার বোতাম। TLabel: পড়ার লেখা, এখানে শুভেচ্ছা দেখা যাবে।

1. Nella palette clicca **TEdit**, poi clicca nella Form1 in alto a sinistra.
প্যালেটে TEdit ক্লিক করো, তারপর Form1-এর উপরে বাঁদিকে ক্লিক করো।

2. Nell'Ispettore oggetti, a sinistra, cancella il testo della proprietà `Text`: la casella resta vuota.
বাঁদিকের Ispettore oggetti-তে Text-এর লেখা মুছে দাও: ঘরটি খালি থাকবে।

3. Nella palette clicca **TButton**, poi clicca nella Form1 sotto la casella.
প্যালেটে TButton ক্লিক করো, তারপর ঘরের নিচে Form1-এ ক্লিক করো।

4. Nell'Ispettore oggetti cambia `Caption` del bottone in:
Ispettore oggetti-তে বোতামের Caption বদলে লেখো:

```
Saluta
```

5. Nella palette clicca **TLabel** (la «A»), poi clicca nella Form1 sotto il bottone.
প্যালেটে TLabel («A») ক্লিক করো, তারপর বোতামের নিচে Form1-এ ক্লিক করো।

6. Nell'Ispettore oggetti cambia `Caption` della label in:
Ispettore oggetti-তে লেবেলের Caption বদলে লেখো:

```
...
```

FATTO! La finestra ha una casella, il bottone «Saluta» e la label «...».
হয়ে গেছে! জানালায় একটি ঘর, «Saluta» বোতাম আর «...» লেবেল আছে।

4. Il codice: il saluto nella Label e nel ShowMessage · কোড: লেবেলে আর ShowMessage-এ শুভেচ্ছা

1. Fai **doppio clic sul bottone** «Saluta». Si apre il codice, dentro la procedura `Button1Click`.
«Saluta» বোতামে দুবার ক্লিক করো। কোড খুলবে, Button1Click প্রসিডিউরের ভিতরে।

2. Tra `begin` e `end;` scrivi le due righe. La procedura completa è questa:
begin আর end;-এর মাঝে দুটি লাইন লেখো। পুরো প্রসিডিউরটি এই রকম:

```
procedure TForm1.Button1Click(Sender: TObject); begin   Label1.Caption := 'Ciao ' + Edit1.Text + '!'
```

Riga	Che cosa fa
`Label1.Caption := 'Ciao ' + Edit1.Text + '!';`	scrive nella **Label** «Ciao», il nome della casella e «!». Il saluto **resta** nella finestra
`ShowMessage('Hello World, ' + Edit1.Text);`	apre una **finestrella** con «Hello World» e il nome, e il bottone OK
`:=`	assegna: «metti dentro». In Pascal si scrive con i due punti
`+`	attacca due scritte una dopo l'altra
`'Ciao '`	una scritta fissa, tra apici singoli. Lo spazio dopo Ciao evita «CiaoAhmed»

প্রথম লাইন: লেবেলে «Ciao» + নাম + «!» লেখে, জানালায় থেকে যায়। দ্বিতীয় লাইন: «Hello World» আর নাম সহ একটি ছোট জানালা খোলে। := মানে «ভিতরে রাখো»। + দুটি লেখা জোড়া দেয়।

5. Salva e fai partire · সেভ করো আর চালাও

1. Premi `Ctrl` + `S`: Lazarus salva tutto nella cartella Saluto.
Ctrl + S চাপো: Lazarus সব Saluto ফোল্ডারে সেভ করে।

2. Premi `F9` (o la freccia verde ▶ in alto). Si apre la TUA finestra.
F9 চাপো (বা উপরের সবুজ ▶)। তোমার জানালা খুলবে।

3. Scrivi il tuo nome nella casella e clicca **Saluta**. Compare la finestrella «Hello World» con il tuo nome.
ঘরে তোমার নাম লেখো আর Saluta চাপো। তোমার নাম সহ «Hello World» ছোট জানালা আসবে।

4. Clicca **OK**: nella Label c'è scritto «Ciao» e il tuo nome.
OK চাপো: লেবেলে «Ciao» আর তোমার নাম লেখা আছে।

5. Prima di chiudere fai lo **screenshot** per la consegna: `Win` + `Shift` + `S`, trascina un rettangolo intorno alla finestra. L'immagine resta in memoria: la incolli nel Documento (scheda 9).
বন্ধ করার আগে স্ক্রিনশট নাও: Win + Shift + S, জানালার চারপাশে আয়তক্ষেত্র টানো। ছবিটি ডকুমেন্টে পেস্ট করবে (কার্ড ৯)।

Errore? È normale, succede a tutti i programmatori. Guarda la finestra Messaggi in basso: dice la riga. I più comuni: manca il `;`; hai scritto `=` invece di `:=`; manca un apice `'`; la riga è fuori da begin ... end.
ভুল? স্বাভাবিক, সব প্রোগ্রামারেরই হয়। নিচের Messaggi জানালা লাইনটি বলে। সাধারণ ভুল: ; নেই, := এর বদলে =, ' নেই, লাইনটি begin ... end-এর বাইরে।

FATTO! Il tuo Hello World in Lazarus funziona.
হয়ে গেছে! Lazarus-এ তোমার Hello World চলছে।

Ripasso veloce dei file (li avete già visti): nella cartella Saluto, dopo F9, ci sono tanti file. Si tengono, si consegnano e vanno su GitHub solo questi 4:
ফাইলের ছোট পুনরাবৃত্তি: F9-এর পরে Saluto ফোল্ডারে অনেক ফাইল থাকে। শুধু এই ৪টি রাখতে, জমা দিতে আর GitHub-এ দিতে হয়:

File	Che cos'è	Tenere?
`Saluto.lpi`	le informazioni e le opzioni del progetto (si riapre con doppio clic)	**SÌ**
`Saluto.lpr`	il programma principale, fa partire tutto	**SÌ**
`unit1.pas`	il TUO codice (Button1Click)	**SÌ**
`unit1.lfm`	com'è fatta la finestra: pezzi, posizioni, Caption	**SÌ**
`.lps` `.res` `.ico` `.exe`	sessione, risorse, icona, programma finito	no: Lazarus li rifà
`lib\` (`.o` `.ppu` `.compiled` `.or`) e `backup\`	file di lavoro del compilatore e copie di sicurezza	no: si rifanno

রাখতে হবে: .lpi .lpr .pas .lfm। বাকি সব (.lps .res .ico .exe, lib\, backup\) Lazarus নিজেই আবার বানায়।

6. Fallo tuo: il saluto in tante lingue (RadioGroup o ComboBox) · নিজের মতো করো: অনেক ভাষায় শুভেচ্ছা

7. Java nel browser, senza installare niente · ব্রাউজারে Java, কিছু ইনস্টল না করে

Java di solito va **installato**. Oggi usiamo **JavaFiddle**: un sito che compila ed esegue Java **dentro il browser**. Non si installa niente e non serve un account.

সাধারণত Java ইনস্টল করতে হয়। আজ আমরা JavaFiddle ব্যবহার করবো: একটি সাইট যা ব্রাউজারের ভিতরেই Java চালায়। কিছু ইনস্টল করতে হয় না।

1. Premi `Ctrl` + `T`: si apre una scheda nuova del browser.

Ctrl + T চাপো: ব্রাউজারে নতুন ট্যাব খুলবে।

2. Scrivi questo indirizzo in alto e premi `Invio`:

উপরে এই ঠিকানাটি লেখো আর Invio চাপো (ঠিকানা, ঠিক এভাবেই লেখো):

```
https://javafiddle.leaningtech.com
```

3. Aspetta: la prima volta il sito carica Java nel browser e ci mette fino a un minuto. A sinistra c'è il riquadro del codice.

অপেক্ষা করো: প্রথমবার সাইটটি ব্রাউজারে Java লোড করে, এক মিনিট পর্যন্ত লাগে। বাঁদিকে কোডের ঘর।

4. Clicca dentro il riquadro del codice, premi `Ctrl` + `A` e poi `Canc`: il riquadro è vuoto.

কোডের ঘরে ক্লিক করো, Ctrl + A তারপর Canc চাপো: ঘরটি খালি।

5. Scrivi questo programma (al posto di Ahmed metti il tuo nome):

এই প্রোগ্রামটি লেখো (Ahmed-এর জায়গায় তোমার নাম):

```
public class Main {     public static void main(String[] args) {         System.out.println("Hello W
```

6. Clicca il bottone **Run** (in alto, «esegui»). Sotto o a destra compare la scritta **Hello World** e il tuo saluto.

Run বোতাম চাপো (উপরে, «চালাও»)। নিচে বা ডানদিকে Hello World আর তোমার শুভেচ্ছা দেখা যাবে।

7. Fai lo **screenshot** con il codice e la scritta: `Win` + `Shift` + `S`.

কোড আর লেখা সহ স্ক্রিনশট নাও: Win + Shift + S।

FATTO! Hai scritto ed eseguito il tuo primo programma Java.
হয়ে গেছে! তুমি তোমার প্রথম Java প্রোগ্রাম লিখে চালিয়েছো।

Piano B. Il sito non si apre o resta bloccato più di 2 minuti? Usa questo, funziona allo stesso modo (il bottone verde **Run** è in alto; la scritta compare in basso):

পরিকল্পনা B: সাইট না খুললে বা ২ মিনিটের বেশি আটকে থাকলে এটা ব্যবহার করো, একইভাবে চলে (উপরে সবুজ Run বোতাম; লেখা নিচে আসে):

```
https://www.onlinegdb.com/online_java_compiler
```

Il nome dopo `public class` resta `Main`: i due siti cercano una classe con questo nome.

public class-এর পরের নাম Main থাকবে: দুটি সাইটই এই নামের ক্লাস খোঁজে।

Per i più veloci: una finestra con un bottone in Java. JavaFiddle sa aprire anche le finestre (Swing). Prova questo programma: confrontalo con le righe che hai scritto in Lazarus.

দ্রুতদের জন্য: Java-তে বোতাম সহ একটি জানালা। JavaFiddle জানালাও (Swing) খুলতে পারে। এই প্রোগ্রামটি চেষ্টা করো, Lazarus-এর লাইনের সাথে তুলনা করো।

```
import javax.swing.*;  public class Main {     public static void main(String[] args) {         JFra
```

8. Java riga per riga, e il confronto con Pascal · Java লাইন ধরে ধরে, Pascal-এর সাথে তুলনা

Riga Java	Che cosa vuol dire
`public class Main {`	in Java tutto sta dentro una **classe**. Si chiama `Main`; la graffa `{` apre il blocco
`public static void main(String[] args) {`	il **metodo main**: il punto da cui parte il programma. Ogni programma Java ne ha uno
`System.out.println("Hello World");`	scrive una riga sullo schermo (la **console**) e va a capo. Il testo sta tra virgolette doppie
`}`	chiude il blocco: la prima `}` chiude main, l'ultima chiude la classe

public class Main: Java-তে সব একটি ক্লাসের ভিতরে থাকে। main: প্রোগ্রাম এখান থেকে শুরু হয়। System.out.println: স্ক্রিনে একটি লাইন লেখে। } ব্লক বন্ধ করে।

Lo stesso programma in Pascal (da console, come lunedì):

একই প্রোগ্রাম Pascal-এ (কনসোল, সোমবারের মতো):

```
program Saluto; begin   writeln('Hello World');   writeln('Ciao, sono Ahmed'); end.
```

	Pascal (Lazarus)	Java
inizio del programma	`program Saluto;` … `begin`	`public class Main {` + `main`
fine	`end.` (con il punto)	`}` e `}`
scrivere una riga	`writeln(…)`	`System.out.println(…)`
finestrella con un messaggio	`ShowMessage(…)`	`JOptionPane.showMessageDialog(null, …)`
le scritte	apici singoli `'Ciao'`	virgolette doppie `"Ciao"`
assegnare	`:=`	`=`
maiuscole e minuscole	uguali: `WriteLn` = `writeln`	diverse: `system.out` è un errore
la finestra	disegnata con il mouse	scritta con il codice (Swing)

Pascal: program … begin … end., writeln, 'একক উদ্ধৃতি', :=, বড়-ছোট হাতের অক্ষর একই। Java: class + main, { }, System.out.println, "দ্বৈত উদ্ধৃতি", =, বড়-ছোট হাতের অক্ষর আলাদা।

Prova del nove: spiega al compagno in 30 secondi che cosa fa `System.out.println` e a che cosa corrisponde in Pascal.

নিজে যাচাই করো: ৩০ সেকেন্ডে সহপাঠীকে বলো System.out.println কী করে আর Pascal-এ এর মতো কী।

9. Consegna · জমা দেওয়া

Su Classroom c'è il compito **«Hello World in Lazarus e in Java»**. Dentro c'è un **Documento Google** già pronto, solo tuo. Aprilo e metti 5 cose:

Classroom-এ «Hello World in Lazarus e in Java» কাজটি আছে। ভিতরে তোমার নিজের একটি Google ডকুমেন্ট আছে। খুলে ৫টি জিনিস দাও:

1. Il codice Lazarus. In Lazarus seleziona la procedura, da `procedure TForm1.Button1Click` fino a `end;`, premi `Ctrl` + `C`; nel Documento clicca sotto «1» e premi `Ctrl` + `V`.

Lazarus কোড: procedure থেকে end; পর্যন্ত বেছে Ctrl + C, ডকুমেন্টে «1»-এর নিচে Ctrl + V।

2. Lo screenshot Lazarus. Programma acceso con il saluto: `Win` + `Shift` + `S`, poi nel Documento sotto «2» `Ctrl` + `V`.

Lazarus স্ক্রিনশট: চালু প্রোগ্রাম, Win + Shift + S, ডকুমেন্টে «2»-এর নিচে Ctrl + V।

3. Il codice Java. In JavaFiddle `Ctrl` + `A` e `Ctrl` + `C` nel riquadro del codice; nel Documento sotto «3» `Ctrl` + `V`.

Java কোড: JavaFiddle-এ কোডের ঘরে Ctrl + A আর Ctrl + C, ডকুমেন্টে «3»-এর নিচে Ctrl + V।

4. Lo screenshot Java con la scritta Hello World: sotto «4».

Hello World লেখা সহ Java স্ক্রিনশট: «4»-এর নিচে।

5. Con parole tue. Sotto «5» scrivi una o due frasi: che differenza c'è tra `ShowMessage` in Lazarus e `System.out.println` in Java? Puoi scrivere anche nella tua lingua.

নিজের ভাষায়: «5»-এর নিচে এক-দুই বাক্য লেখো: Lazarus-এর ShowMessage আর Java-র System.out.println-এর পার্থক্য কী? নিজের ভাষাতেও লিখতে পারো।

6. In Classroom premi **Consegna** (il bottone blu in alto a destra nel compito), entro le **14:00**.

Classroom-এ «Consegna» চাপো (কাজের উপরে ডানদিকে নীল বোতাম), ১৪:০০-এর মধ্যে।

Il tuo cruscotto (scheda 1) ti dice che cosa manca nel Documento. Se manca qualcosa: «Annulla consegna», completa, «Consegna».

তোমার ড্যাশবোর্ড (কার্ড ১) বলে ডকুমেন্টে কী বাকি। কিছু বাকি থাকলে: «Annulla consegna», সম্পূর্ণ করো, আবার «Consegna»।

Le schede della lezione

1. Oggi: Java con Eclipse, senza installare niente · আজ: Eclipse দিয়ে Java, কিছু ইনস্টল না করে

LA GUIDA CON LE FOTO: Eclipse passo per passo, fino al bottone Hello World

ছবিসহ গাইড: ধাপে ধাপে Eclipse, Hello World বোতাম পর্যন্ত

Questa pagina cambia durante la lezione: ogni tanto premi F5 per vedere la versione nuova.
এই পাতাটি ক্লাসের সময় বদলায়: মাঝে মাঝে F5 চাপো, নতুন সংস্করণ দেখতে।

Oggi scrivi il tuo primo programma Java in un programma vero da programmatori: **Eclipse**.

আজ তুমি প্রোগ্রামারদের আসল প্রোগ্রাম Eclipse-এ তোমার প্রথম Java প্রোগ্রাম লিখবে।

Eclipse è un **IDE** (si legge «ài-di-ì»), cioè un **ambiente di sviluppo**: un unico programma che contiene tre cose.

Eclipse একটি IDE, মানে উন্নয়নের পরিবেশ: একটি প্রোগ্রামের ভিতরে তিনটি জিনিস।

Pezzo	Che cosa fa	In Lazarus
1. Editor	il foglio dove scrivi il codice	la finestra del codice (unit1.pas)
2. Compilatore	traduce il codice nella lingua del computer e trova gli errori	Compila
3. Esecuzione	fa partire il programma	il triangolo verde Esegui (F9)

১. Editor: কোড লেখার পাতা। ২. Compiler: কোড কম্পিউটারের ভাষায় অনুবাদ করে আর ভুল খোঁজে। ৩. চালানো: প্রোগ্রাম শুরু করে। Lazarus-ও একটি IDE।

Anche **Lazarus è un IDE**: tu lo conosci già. Eclipse è la stessa idea, ma per Java.

Lazarus-ও একটি IDE: তুমি এটা চেনো। Eclipse একই ধারণা, কিন্তু Java-র জন্য।

Eclipse qui è **portabile**: non si installa. È una cartella dentro un file ZIP (un pacchetto compresso): la apri («scompatti») e fai doppio clic sul programma. Dentro c'è già anche Java. Per questo funziona a scuola senza chiedere niente al tecnico.

এখানে Eclipse পোর্টেবল: ইনস্টল করতে হয় না। এটি একটি ZIP ফাইলের ভিতরে একটি ফোল্ডার: খুলে (আনজিপ করে) প্রোগ্রামে দুবার ক্লিক করো। ভিতরে Java-ও আছে।

Eclipse è **in inglese**. In questa pagina trovi il nome inglese esatto di ogni menu e bottone, e accanto la traduzione italiana tra parentesi.
Eclipse ইংরেজিতে। এই পাতায় প্রতিটি মেনু আর বোতামের সঠিক ইংরেজি নাম আছে, পাশে বন্ধনীতে ইতালীয় অনুবাদ।

Parte	Che cosa fai	Schede
A. Preparare	prendi lo ZIP, scompattalo, apri Eclipse	2, 3, 4
B. Programmare	il progetto HelloWorld e la prova a console	5, 6
C. LA META	**una finestra con un bottone: lo premi e compare «Hello World»**	7, 8, 9
D. Consegna	lo screenshot nel Documento su Classroom, entro le 14:00	10
E. Per chi ha finito	il saluto con il tuo nome (facoltativo)	11

A: ZIP নাও, আনজিপ করো, Eclipse খোলো। B: HelloWorld প্রজেক্ট আর কনসোলে পরীক্ষা। C: লক্ষ্য: বোতামসহ জানালা, চাপলে «Hello World» আসে। D: Classroom-এ স্ক্রিনশট, ১৪:০০-এর মধ্যে। E: যারা শেষ করেছে: তোমার নাম দিয়ে শুভেচ্ছা।

2. La mappa dello schermo di Eclipse · Eclipse-এর পর্দার মানচিত্র

Prima di cominciare, guarda com'è fatta la finestra di Eclipse. Ti servono solo **5 zone**.

শুরু করার আগে Eclipse-এর জানালাটা দেখো। শুধু ৫টি জায়গা দরকার।

```
+-------------------------------------------------------------+ | File  Edit  Source  Refactor  Navig
```

Dove	Nome in Eclipse	A che cosa serve
in alto, la prima riga di parole	la barra dei menu: **«File»** (File), **«Run»** (Esegui)…	i comandi, come in Lazarus
in alto, sotto i menu	la barra dei bottoni; il **cerchio verde con il triangolo bianco** è **«Run»** (Esegui)	fa partire il programma
a sinistra	**«Package Explorer»** (Esplora pacchetti)	l'elenco dei tuoi progetti e dei tuoi file
al centro, la parte grande	l'**editor**	qui scrivi il codice
in basso	la scheda **«Console»** (Console)	qui compare quello che il programma scrive

উপরে: মেনু (File, Run)। মেনুর নিচে: বোতাম; সবুজ বৃত্তে সাদা ত্রিভুজ = Run (চালাও)। বামে: Package Explorer (তোমার প্রজেক্ট আর ফাইল)। মাঝখানে: editor (কোড লেখো)। নিচে: Console (প্রোগ্রাম যা লেখে)।

Il triangolo verde lo conosci già: in Lazarus fa la stessa cosa.

সবুজ ত্রিভুজ তুমি চেনো: Lazarus-এও একই কাজ করে।

3. Prendi lo ZIP e scompattalo in una cartella corta · ZIP নাও আর ছোট নামের ফোল্ডারে আনজিপ করো

Il prof ti manda il file **eclipse-java-2026-09-R-win32-x86_64.zip** (360 MB) sul tuo computer.

প্রফেসর তোমার কম্পিউটারে eclipse-java-2026-09-R-win32-x86_64.zip ফাইলটি (৩৬০ MB) পাঠাবেন।

1. Apri **Esplora file**: l'icona della **cartella gialla** nella barra in basso dello schermo (oppure premi `Win` + `E`).

Esplora file খোলো: স্ক্রিনের নিচের বারে হলুদ ফোল্ডারের আইকন (বা Win + E)।

2. Il file è nella tua cartella personale. Per trovarlo: in Esplora file clicca nella **casella di ricerca in alto a destra**, scrivi `eclipse-java` e aspetta che compaia il file con la cerniera (l'icona dello ZIP).

ফাইলটি তোমার ব্যক্তিগত ফোল্ডারে। Esplora file-এ উপরে ডানদিকে খোঁজার ঘরে ক্লিক করো, eclipse-java লেখো, চেনওয়ালা (ZIP) ফাইল আসা পর্যন্ত অপেক্ষা করো।

Se il prof ti dice di prenderlo dal disco **P:**, nel pannello a sinistra di Esplora file clicca **P:** e poi la cartella che ti indica.

প্রফেসর P: ডিস্ক থেকে নিতে বললে, Esplora file-এর বামে P: ক্লিক করো, তারপর যে ফোল্ডার বলবেন।

3. Clic **destro** sul file ZIP ⟶ **Estrai tutto…**

ZIP ফাইলে ডান ক্লিক ⟶ Estrai tutto…

4. Clicca il bottone verde **Copia** qui sotto: è una cartella dal nome corto.

নিচের সবুজ Copia বোতামে ক্লিক করো: ছোট নামের একটি ফোল্ডার।

```
C:\Users\Public\ec
```
Copia

5. Nella finestra di Estrai tutto clicca nella casella del percorso (la riga lunga con `C:\Users\…`), premi `Ctrl` + `A` e poi `Ctrl` + `V`.

Estrai tutto জানালায় পথের ঘরে ক্লিক (C:\Users\... লেখা লম্বা লাইন), Ctrl + A তারপর Ctrl + V।

Perché una cartella corta: Eclipse contiene file con nomi lunghissimi. Se il percorso è troppo lungo, Windows non riesce a scompattarli tutti ed Eclipse non parte.
ছোট ফোল্ডার কেন: Eclipse-এ খুব লম্বা নামের ফাইল আছে। পথ বেশি লম্বা হলে Windows সব আনজিপ করতে পারে না, Eclipse চালু হয় না।

6. Clicca il bottone **Estrai** in basso a destra.

নিচে ডানদিকে Estrai বোতামে ক্লিক করো।

Dura qualche minuto (sono migliaia di file): la barra verde avanza piano. È normale: aspetta che la finestra si chiuda da sola e non chiudere niente.
কয়েক মিনিট লাগে (হাজার হাজার ফাইল): সবুজ বার ধীরে এগোয়। স্বাভাবিক: জানালা নিজে বন্ধ হওয়া পর্যন্ত অপেক্ষা করো।

Se Windows dice che non puoi scrivere in `C:\Users\Public`: rifai i passi 3-6 e incolla invece questo percorso, poi sostituisci **nome.cognome** con il tuo:

C:\Users\Public-এ লিখতে না দিলে: ৩-৬ ধাপ আবার করো, এই পথ পেস্ট করো, nome.cognome-এর জায়গায় তোমারটা লেখো:

```
C:\Users\nome.cognome\Documents\ec
```
Copia

7. Alla fine Windows apre la cartella **ec**. Dentro c'è la cartella **eclipse**: doppio clic per entrare.

শেষে Windows ec ফোল্ডার খুলবে। ভিতরে eclipse ফোল্ডার: দুবার ক্লিক করে ঢোকো।

8. Controllo: dentro **eclipse** devono esserci le tre cartelle **configuration**, **features** e **plugins**.

যাচাই: eclipse-এর ভিতরে তিনটি ফোল্ডার থাকতে হবে: configuration, features, plugins।

Se le vedi solo 6 cose e mancano **plugins** e **features**: l'estrazione non è finita. Chiudi Esplora file, cancella la cartella **ec** e rifai dal passo 3, aspettando la fine.
যদি শুধু ৬টি জিনিস থাকে আর plugins, features না থাকে: আনজিপ শেষ হয়নি। ec ফোল্ডার মুছে ৩ নম্বর ধাপ থেকে আবার করো, শেষ পর্যন্ত অপেক্ষা করো।

FATTO! Hai la cartella `C:\Users\Public\ec\eclipse` con dentro plugins. Il programma è pronto, senza installare niente.
হয়ে গেছে! ec\eclipse ফোল্ডারে plugins আছে। কিছু ইনস্টল না করেই প্রোগ্রাম তৈরি।

4. Primo avvio di Eclipse · Eclipse প্রথমবার চালু করা

1. Nella cartella **ec\eclipse** cerca il file **eclipse.exe**: ha l'icona di un **cerchio viola** (a volte si vede solo «eclipse», senza «.exe»). Doppio clic.

eclipse ফোল্ডারে eclipse.exe খোঁজো: বেগুনি বৃত্তের আইকন (কখনো শুধু «eclipse» দেখায়)। দুবার ক্লিক।

Se compare la finestra blu **«Windows ha protetto il PC»**: clicca la scritta **Ulteriori informazioni**, poi il bottone **Esegui comunque**.
নীল জানালা «Windows ha protetto il PC» এলে: Ulteriori informazioni লেখায় ক্লিক, তারপর Esegui comunque বোতাম।

Se Windows dice che l'app è **bloccata dall'amministratore**: fermati e chiama il prof. Non è colpa tua: oggi si userà Java nel sito web.
Windows যদি বলে অ্যাপটি অ্যাডমিন ব্লক করেছে: থামো আর প্রফেসরকে ডাকো। তোমার দোষ নয়: আজ ওয়েবসাইটে Java ব্যবহার হবে।

2. Aspetta: Eclipse si apre piano la prima volta (anche un minuto).

অপেক্ষা করো: প্রথমবার Eclipse ধীরে খোলে (এক মিনিট পর্যন্ত)।

3. Compare la finestra **«Select a directory as workspace»** (Scegli una cartella come area di lavoro). Il **workspace** è la cartella dove Eclipse salva i tuoi progetti. Non cambiare niente: clicca il bottone **«Launch»** (Avvia) in basso a destra.

«Select a directory as workspace» জানালা আসবে। workspace হলো যে ফোল্ডারে Eclipse তোমার প্রজেক্ট রাখে। কিছু বদলিও না: নিচে ডানদিকে Launch ক্লিক করো।

4. Si apre la pagina **«Sponsor the Eclipse IDE»** (Sostieni Eclipse) con dei bottoni arancioni. Clicca il bottone arancione **«Later»** (Più tardi).

«Sponsor the Eclipse IDE» পাতা খুলবে, কমলা বোতামসহ। কমলা Later (পরে) বোতামে ক্লিক করো।

5. In alto ci sono altre linguette (schede) aperte, per esempio **«Install Java 27 Support»** (Installa il supporto a Java 27) e **«Eclipse IDE for Java Developers 2026-12 M1»** (notizie sulla prossima versione). Non servono: chiudile una alla volta con la accanto al loro nome.

উপরে আরও ট্যাব খোলা আছে, যেমন Install Java 27 Support আর Eclipse IDE for Java Developers 2026-12 M1। দরকার নেই: নামের পাশের দিয়ে একটা একটা করে বন্ধ করো।

6. Ora rimetti in ordine le zone dello schermo. In alto clicca il menu **«Window»** (Finestra).

এখন পর্দার জায়গাগুলো ঠিক করো। উপরে Window (জানালা) মেনুতে ক্লিক করো।

7. Porta il mouse su **«Perspective»** (Vista) e clicca **«Reset Perspective...»** (Ripristina la vista).

Perspective-এর উপর মাউস রাখো, Reset Perspective... ক্লিক করো।

8. Nella finestrella clicca il bottone **«Reset»** (Ripristina). Ora a sinistra compare il riquadro **«Package Explorer»** (Esplora pacchetti).

ছোট জানালায় Reset বোতামে ক্লিক করো। এখন বামে Package Explorer দেখা যাবে।

Se a sinistra vedi ancora solo icone piccole: menu **«Window»** (Finestra) ⟶ **«Show View»** (Mostra vista) ⟶ **«Package Explorer»** (Esplora pacchetti).
বামে এখনো শুধু ছোট আইকন থাকলে: Window ⟶ Show View ⟶ Package Explorer।

Ora vedi lo schermo della mappa (scheda 2): a sinistra **«Package Explorer»** (Esplora pacchetti), ancora vuoto.

এখন মানচিত্রের পর্দা দেখছো (কার্ড ২): বামে Package Explorer, এখনো খালি।

FATTO! Eclipse è aperto.
হয়ে গেছে! Eclipse খোলা হয়েছে।

5. Il progetto nuovo «HelloWorld» · নতুন প্রজেক্ট «HelloWorld»

Come in Lazarus: **un progetto = una cartella**. Eclipse la crea da solo nel workspace.

Lazarus-এর মতো: একটি প্রজেক্ট = একটি ফোল্ডার। Eclipse নিজেই workspace-এ বানায়।

1. A sinistra, nel riquadro **«Package Explorer»** (Esplora pacchetti), clicca la scritta blu **«Create a Java project»** (Crea un progetto Java).

বামে Package Explorer-এ নীল লেখা Create a Java project-এ ক্লিক করো।

Se la scritta blu non c'è: in alto a sinistra menu **«File»** (File) ⟶ **«New»** (Nuovo) ⟶ **«Java Project»** (Progetto Java).

নীল লেখা না থাকলে: উপরে বামে File ⟶ New ⟶ Java Project।

2. Nella casella in alto **«Project name»** (Nome del progetto) scrivi (tutto attaccato):

উপরের Project name ঘরে লেখো (একসাথে):

```
HelloWorld
```
Copia

3. Scendi in fondo alla finestra, nella parte **«Module»** (Modulo): **togli la spunta** a **«Create module-info.java file»** (Crea il file module-info.java).

জানালার নিচে Module অংশে: Create module-info.java file-এর টিক তুলে দাও।

Perché: quel file serve per i programmi grandi, divisi in tanti pezzi; per il nostro programma piccolo dà solo fastidio.

কারণ: ওই ফাইল বড় প্রোগ্রামের জন্য; আমাদের ছোট প্রোগ্রামে শুধু ঝামেলা করে।

4. Clicca il bottone **«Finish»** (Fine) in basso a destra.

নিচে ডানদিকে Finish ক্লিক করো।

Se compare una domanda su **«Open Associated Perspective?»** (Aprire la vista collegata?): clicca **«Open Perspective»** (Apri la vista).

«Open Associated Perspective?» প্রশ্ন এলে: Open Perspective ক্লিক করো।

FATTO! A sinistra, nel Package Explorer, c'è il progetto **HelloWorld**.
হয়ে গেছে! বামে Package Explorer-এ HelloWorld প্রজেক্ট আছে।

6. La classe Main e la prova veloce: Hello World nella Console · Main ক্লাস আর দ্রুত পরীক্ষা: Console-এ Hello World

1. A sinistra, nel Package Explorer, clicca la **freccina >** accanto a **HelloWorld**: compare la cartella **src** (da «source», il codice).

বামে HelloWorld-এর পাশের > তীরে ক্লিক করো: src ফোল্ডার দেখা যাবে (source = কোড)।

2. Clic **destro** su **src** ⟶ **«New»** (Nuovo) ⟶ **«Class»** (Classe).

src-এ ডান ক্লিক ⟶ New ⟶ Class।

3. Nella finestra, casella **«Package»** (Pacchetto): cliccaci dentro, premi `Ctrl` + `A` e poi `Canc`: deve restare **vuota**. Compare un avviso giallo: va bene così.

Package ঘরে ক্লিক, Ctrl + A তারপর Canc: ঘরটি খালি থাকবে। হলুদ সতর্কবার্তা আসবে: ঠিক আছে।

4. Nella casella **«Name»** (Nome) scrivi (con la M maiuscola):

Name ঘরে লেখো (বড় হাতের M):

```
Main
```
Copia

5. Più sotto metti la spunta a **«public static void main(String[] args)»** (crea il punto di partenza main).

নিচে public static void main(String[] args)-এ টিক দাও।

6. Clicca **«Finish»** (Fine) in basso a destra. Al centro, nell'editor, si apre il file **Main.java**.

Finish ক্লিক করো। মাঝখানে editor-এ Main.java খুলবে।

7. Clicca **Copia** qui sotto.

নিচে Copia ক্লিক করো।

```
public class Main {     public static void main(String[] args) {         System.out.println("
```
Copia

8. Clicca dentro l'editor (al centro), premi `Ctrl` + `A` (seleziona tutto), poi `Ctrl` + `V` (incolla), poi `Ctrl` + `S` (salva).

editor-এ ক্লিক, Ctrl + A (সব বাছো), Ctrl + V (পেস্ট), Ctrl + S (সেভ)।

9. In alto clicca il **cerchio verde con il triangolo bianco «Run»** (Esegui). Oppure `Ctrl` + `F11`. Se compare **«Save and Launch»** (Salva e avvia): **«OK»** (OK).

উপরে সবুজ বৃত্তে সাদা ত্রিভুজ Run। বা Ctrl + F11। Save and Launch এলে: OK।

10. Guarda **in basso**, nella scheda **«Console»** (Console): c'è scritto **Hello World**.

নিচে Console ট্যাবে দেখো: Hello World লেখা আছে।

FATTO! Run funziona: Eclipse compila ed esegue. Ora la meta vera: la finestra con il bottone (scheda 7).
হয়ে গেছে! Run কাজ করছে। এখন আসল লক্ষ্য: বোতামসহ জানালা (কার্ড ৭)।

7. LA META: una finestra con un bottone che scrive «Hello World» · লক্ষ্য: বোতামসহ জানালা যা «Hello World» লেখে

Ora fai un programma con la finestra, come quelli di Lazarus: un **bottone** e una **scritta**. Premi il bottone e la scritta diventa **Hello World**. In Java la finestra non si disegna con il mouse: si scrive con il codice (si chiama **Swing**).

এখন Lazarus-এর মতো জানালাসহ প্রোগ্রাম বানাও: একটি বোতাম আর একটি লেখা। বোতাম চাপলে লেখাটি Hello World হয়। Java-তে জানালা মাউসে আঁকা হয় না: কোডে লেখা হয় (Swing)।

1. Clicca **Copia** qui sotto.

নিচে Copia ক্লিক করো।

```
import javax.swing.*;  public class Main {     public static void main(String[] args) {
```

Copia

2. Clicca dentro l'editor di **Main.java** (al centro), `Ctrl` + `A`, `Ctrl` + `V`, `Ctrl` + `S`.

Main.java-র editor-এ ক্লিক, Ctrl + A, Ctrl + V, Ctrl + S।

3. Clicca il triangolo verde **«Run»** (Esegui) in alto.

উপরে সবুজ Run ক্লিক করো।

4. Si apre una finestrella nuova «Il mio primo programma» (può comparire dietro Eclipse: guarda nella barra in basso dello schermo). Clicca **Premi qui**.

নতুন ছোট জানালা «Il mio primo programma» খুলবে (Eclipse-এর পিছনে থাকতে পারে: স্ক্রিনের নিচের বারে দেখো)। Premi qui ক্লিক করো।

FATTO! Accanto al bottone compare **Hello World**. Il tuo primo programma Java con la finestra!
হয়ে গেছে! বোতামের পাশে Hello World দেখা যাবে। জানালাসহ তোমার প্রথম Java প্রোগ্রাম!

5. Fallo tuo: cambia il titolo «Il mio primo programma» e la scritta del bottone «Premi qui» con parole tue (lascia le virgolette `"`). Salva e Run di nuovo.

নিজের মতো করো: «Il mio primo programma» শিরোনাম আর «Premi qui» বোতামের লেখা বদলাও (" রেখে দাও)। সেভ করে আবার Run।

Per chiudere il programma: la **X** della finestrella.

প্রোগ্রাম বন্ধ করতে: ছোট জানালার X।

8. Riga per riga, e il confronto con Lazarus · লাইন ধরে ধরে, Lazarus-এর সাথে তুলনা

```
import javax.swing.*;  public class Main {     public static void main(String[] args) {        JFra
```

Riga	Che cosa vuol dire	In Lazarus
`import javax.swing.*;`	prende i pezzi per fare le finestre	`uses` in cima alla unit
`public class Main {`	tutto sta dentro una **classe**; il nome **Main** è uguale al file Main.java	la unit
`public static void main(…) {`	il punto di partenza: il programma comincia da qui	il file .lpr
`new JFrame("…")`	crea la finestra con il suo titolo	il Form, con la sua Caption
`new JButton("Premi qui")`	crea il bottone	TButton
`new JLabel("...")`	crea la scritta	TLabel
`bottone.addActionListener(e -> …);`	che cosa fare quando premi il bottone	il doppio clic sul bottone: Button1Click
`scritta.setText("Hello World")`	cambia il testo della scritta	`Label1.Caption := 'Hello World';`
`JPanel` e `add(…)`	mette bottone e scritta nella finestra	trascinare i componenti sul Form
`setSize(300, 120)`	larghezza e altezza della finestra	Width e Height
`setDefaultCloseOperation(…)`	con la X il programma si chiude davvero	automatico
`setVisible(true)`	mostra la finestra	automatico
`;` e `{ }`	fine istruzione; apri e chiudi un blocco	`;` e `begin … end`

import: জানালার অংশ আনে (uses)। class Main: ক্লাস, নাম = ফাইল Main.java। main: শুরু। JFrame = Form। JButton = TButton। JLabel = TLabel। addActionListener = Button1Click। setText = Label1.Caption :=। add = Form-এ রাখা। setSize = Width, Height। setVisible(true) = জানালা দেখায়। { } = begin … end।

	Pascal	Java
scrivi una riga in console	`writeln(…)`	`System.out.println(…)`
le scritte	`'Ciao'`	`"Ciao"`
assegnare	`:=`	`=` oppure `setText(…)`
maiuscole e minuscole	uguali	**diverse**: `jbutton` è un errore

Prova del nove: spiega al compagno in 30 secondi che cosa succede quando premi il bottone, e a che cosa corrisponde in Lazarus.

নিজে যাচাই: ৩০ সেকেন্ডে সহপাঠীকে বলো বোতাম চাপলে কী হয়, আর Lazarus-এ এর মতো কী।

9. Errori comuni: l'errore è normale · সাধারণ ভুল: ভুল হওয়া স্বাভাবিক

Sbagliare succede a tutti i programmatori, anche ai professionisti. Eclipse ti aiuta: sottolinea l'errore in **rosso** e mette una **X rossa** a sinistra della riga. Passa il mouse sulla X per leggere il messaggio.

ভুল সব প্রোগ্রামারের হয়, পেশাদারদেরও। Eclipse ভুলে লাল দাগ দেয় আর লাইনের বামে লাল X দেয়। বার্তা পড়তে X-এর উপর মাউস রাখো।

Che cosa vedi	Perché	Come si sistema
`The Eclipse executable launcher was unable to locate its companion shared library`	lo ZIP non è stato scompattato tutto: manca la cartella **plugins** (percorso troppo lungo o estrazione interrotta)	chiudi, cancella la cartella, rifai la scheda 3 con la cartella corta `C:\Users\Public\ec` e aspetta la fine
rosso sotto `system` o `jbutton`	Java distingue maiuscole e minuscole	copia di nuovo il codice con il bottone Copia
`Syntax error, insert ";"`	manca il punto e virgola	aggiungi `;` alla fine della riga
`The public type … must be defined in its own file`	il nome dopo `class` è diverso dal nome del file	dopo `public class` scrivi `Main`, come il file Main.java
rosso sulle virgolette	una virgoletta `"` manca	il testo sta tra due `"`
il Run è **grigio** o chiede che cosa avviare	manca `main`, oppure il cursore non è nel file Main.java	clicca dentro Main.java nell'editor e riprova
premi Run e la finestrella non si vede	è dietro Eclipse	guarda nella barra in basso dello schermo e cliccala
la Console non si vede	la scheda è nascosta	menu **«Window»** (Finestra) ⟶ **«Show View»** (Mostra vista) ⟶ **«Console»** (Console)

companion shared library: ZIP পুরো আনজিপ হয়নি (plugins নেই): ফোল্ডার মুছে কার্ড ৩ আবার করো, ছোট ফোল্ডার C:\Users\Public\ec দিয়ে। লাল system/jbutton: বড়-ছোট অক্ষর; কোড আবার কপি করো। insert ";": ; দাও। must be defined in its own file: class-এর পরে Main। Run ধূসর: Main.java-তে ক্লিক করো। জানালা দেখা যায় না: নিচের বারে দেখো। Console নেই: Window ⟶ Show View ⟶ Console।

Per tornare indietro: `Ctrl` + `Z` annulla l'ultima modifica.

পিছনে যেতে: Ctrl + Z শেষ পরিবর্তন বাতিল করে।

10. Consegna · জমা দেওয়া

Su Classroom c'è il compito di oggi **«Hello World in Lazarus e in Java»**, con il tuo Documento Google.

Classroom-এ আজকের কাজ «Hello World in Lazarus e in Java», তোমার Google ডকুমেন্টসহ।

1. Apri il Documento del compito su Classroom (nella finestra del browser).

Classroom-এ কাজের ডকুমেন্ট খোলো (ব্রাউজারে)।

2. Torna su Eclipse: fai partire il programma e premi il bottone, così si vede **Hello World** nella finestrella.

Eclipse-এ ফিরে যাও: প্রোগ্রাম চালাও, বোতাম চাপো, যাতে ছোট জানালায় Hello World দেখা যায়।

3. Premi `Win` + `Shift` + `S` e trascina il mouse attorno a Eclipse **e** alla finestrella con Hello World.

Win + Shift + S চাপো, Eclipse আর Hello World লেখা ছোট জানালার চারপাশে মাউস টানো।

4. Nel Documento clicca sotto «4» (lo screenshot Java): se c'è già uno screenshot Java, cliccalo e premi `Canc`. Poi premi `Ctrl` + `V`.

ডকুমেন্টে «4»-এর নিচে ক্লিক করো: আগের Java স্ক্রিনশট থাকলে সেটিতে ক্লিক করে Canc। তারপর Ctrl + V।

5. In Classroom premi **Consegna** (il bottone blu in alto a destra nel compito), entro le **14:00**. Se l'avevi già consegnato: prima **Annulla consegna**, poi incolla, poi di nuovo **Consegna**.

Classroom-এ Consegna চাপো (উপরে ডানদিকে নীল বোতাম), ১৪:০০-এর মধ্যে। আগে জমা দিয়ে থাকলে: আগে Annulla consegna, তারপর পেস্ট, তারপর আবার Consegna।

FATTO! Hai consegnato il tuo programma Java con la finestra, fatto con Eclipse.
হয়ে গেছে! Eclipse দিয়ে বানানো জানালাসহ তোমার Java প্রোগ্রাম জমা দিয়েছো।

11. Per chi ha finito: saluta il tuo nome (facoltativo) · যারা শেষ করেছে: শুভেচ্ছা জানানো জানালা (ঐচ্ছিক)

Lo stesso programma «Ciao» che la 1INF ha fatto oggi in Lazarus, ma in Java: una casella per il nome, un bottone, un messaggio. Qui la finestra non si disegna con il mouse: si scrive con il codice (si chiama **Swing**).

1INF আজ Lazarus-এ যে «Ciao» প্রোগ্রাম বানিয়েছে, সেটাই Java-তে: নামের ঘর, বোতাম, বার্তা। এখানে জানালা মাউসে আঁকা হয় না: কোডে লেখা হয় (Swing)।

1. A sinistra clic destro su **src** ⟶ **«New»** (Nuovo) ⟶ **«Class»** (Classe).

বামে src-এ ডান ক্লিক ⟶ New ⟶ Class।

2. Casella **«Package»** (Pacchetto): `Ctrl` + `A` e `Canc` (vuota). Casella **«Name»** (Nome):

Package: Ctrl + A আর Canc (খালি)। Name ঘরে:

```
Finestra
```
Copia

3. Clicca **«Finish»** (Fine). Poi clicca **Copia** qui sotto.

Finish ক্লিক করো। তারপর নিচে Copia।

```
import javax.swing.*;  public class Finestra {     public static void main(String[] args) {
```
Copia

4. Nell'editor: `Ctrl` + `A`, `Ctrl` + `V`, `Ctrl` + `S`. Poi il triangolo verde **«Run»** (Esegui).

editor-এ: Ctrl + A, Ctrl + V, Ctrl + S। তারপর সবুজ Run।

5. Scrivi il tuo nome nella casella e premi **Saluta**.

ঘরে তোমার নাম লেখো আর Saluta চাপো।

Java (Swing)	Lazarus
`JFrame`	il Form (la finestra)
`JTextField`	TEdit (la casella di testo)
`JButton`	TButton (il bottone)
`addActionListener(e -> …)`	il doppio clic sul bottone: Button1Click
`casella.getText()`	`Edit1.Text`
`JOptionPane.showMessageDialog(null, …)`	`ShowMessage(…)`
`JPanel` + `add(…)`	mettere i componenti sul Form
`setVisible(true)`	mostra la finestra

JFrame = Form, JTextField = TEdit, JButton = TButton, addActionListener = Button1Click, getText() = Edit1.Text, showMessageDialog = ShowMessage, setVisible(true) = জানালা দেখায়।

Fallo tuo: cambia il titolo «Il mio saluto» e la parola «Ciao» con quello che vuoi tu.

নিজের মতো করো: «Il mio saluto» শিরোনাম আর «Ciao» শব্দ বদলে দাও।

2 Changelog

1. **v1.0 (29/10/2026)**: prima versione completa: le 16 lezioni dal 23/09 al 28/10, ognuna con la teoria e l’esercitazione.